

LX4x80 Data Sheet

April 7, 2000

Revision 4.0

This document is proprietary and confidential to Lexra Inc.
Copyright © 2000 Lexra Inc.

LEXRA, Inc.

2 University Park, 51 Sawyer Road
Waltham, MA 02154
Tel: 781-899 5799
Fax: 781-899 5769

LEXRA, INC. CONFIDENTIAL

I

LX4x80 Data Sheet

Revision 4.0

April 7, 2000

LEXRA, Inc.

2 University Park, 51 Sawyer Road
Waltham, MA 02154
Tel: 781-899 5799
Fax: 781-899 5769

LEXRA, INC. CONFIDENTIAL

III

This document is proprietary and confidential to Lexra Inc.
Copyright © 2000 Lexra Inc.
ALL RIGHTS RESERVED

MIPS, MIPS16, MIPS ABI, MIPSIV, MIPSV, R3000, R4000, and other MIPS common law marks are trademarks and/or registered trademarks of MIPS Technologies, Inc. Lexra, Inc. is not associated with MIPS Technologies, Inc. in any way.

SmoothCore is a trademark of Lexra, Inc

Table of Contents

Table of Contents	v
LX4x80 Product Brief	1
1.1 Overview	1
1.2 Implementation Overview	4
1.3 RTL Core & SmoothCore™	4
1.4 System Level Building Blocks	5
1.5 Building Block Integration	7
1.6 Software Tool Support	7
1.7 EDA Tool Support	8
LX4x80 Overview	9
2.1 LX4x80 Key Features	2
2.2 Processor Core	2
2.3 System Building Blocks	4
2.4 Design Integration	6
2.5 Reference Specification	6
2.6 Product Delivery	7
The LX4x80 Programming Model	8
3.1 Hardware Architecture of the LX4x80 Core	8
3.2 Summary of LX4x80 Instructions	4
3.3 Opcode Extension Using the Custom Engine Interface (CEI)	11
3.4 Memory Management	13
3.5 Exception Processing	13
3.6 The Coprocessor Interface (CI)	18
3.7 Power Savings Mode	20
MIPS16	21
4.1 MIPS16 Instructions	22
4.2 Mode switching	26
4.3 Exceptions	27
4.4 No Delay Slots	27
4.5 Effect on System Clock	28
Local Memory	29

5.1	Cache Control Register: CCTL	31
5.2	Instruction Cache (ICACHE) LMI	32
5.3	Fill-Once Instruction RAM (IRAM) LMI	34
5.4	Direct Mapped Write Through Data Cache (DCACHE) LMI	35
5.5	Scratch Pad Data RAM (DRAM) LMI	36
The Lexra System Bus		38
EJTAG		39
Customizable Interfaces		40
Multiply-Divide-Accumulate		42
9.1	Summary of Features	42
9.2	High-Lights	43
9.3	Op-codes for standard mode (32-bit) instructions	43
9.4	Op-codes for MIPS-16 (16 bit) mode instructions:	44
9.5	Multiplier Pipelining	46
9.6	Accessing HI and LO after multiply instructions	46
9.7	Divider Overview and Register Usage	47
LX4280 Enhancements and Differences		49
10.1	Instruction Set	49
10.2	Pipeline	55
10.3	EJTAG	58
10.4	Cache	59
10.5	Improved DIV(U) Performance	60

Chapter

1

LX4x80 Product Brief

High Performance 32-Bit Embedded Processor

1.1 Overview

LX4x80 are embedded RISC processor cores targeted at system-on-silicon designers who require the best in price/performance and ease-of-use. The LX4x80 family of processors consist of the single-issue LX4180 and the dual-issue LX4280. Key applications include data communication products such as network protocol processors, cable modems, set-top boxes, as well as disk controllers.

In these high-volume products, cost and time-to-market are the two most critical issues for a design team. The LX4x80 addresses both concerns. The less-than-2.5 mm² LX4180 die size is substantially smaller than most MIPS R3000 based solutions, while the LX4280 275 MIPS delivers enough performance so that designers do not have to resort to expensive MIPS R4000 cores, or complicated multi-processing schemes. The time-to-market advantage for the LX4x80 comes from Lexra's proprietary RISC implementation architecture as well as a 100% ASIC based design methodology. The result is a simple system level interface, complete design kit for EDA software, and flexibility and expandability for the design team.

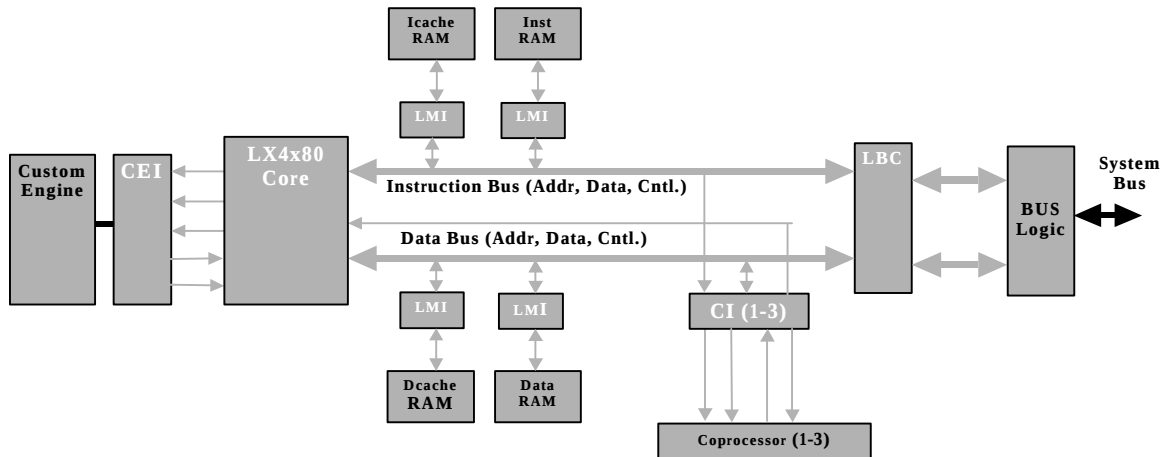
The features of the LX4x80 include:

- **High Performance:** Up to 200 Dhrystone MIPS at 200 MHz in 0.18μ process for the LX4180 and 275 Dhrystone MIPS for the LX4280.
- **Small Die Size:** less than 2.5 mm² in most 0.18 μm processes for the LX4180 and 5.5 mm² for the LX4280.

- **Easy ASIC Design:** Single phase clocking, fully synchronous design, with easy to interface system bus protocol. Supports popular EDA tools.
- **Executes MIPS-I Instruction Set: Execution of** MIPS-I instruction set minimizes the porting effort for the third party software development tools.
- **Easy Customization:** User-defined cache, RAM, addressing and other settings. Interfaces for adding application-specific instructions.

1.1.1 Architecture Overview

The LX4x80 is a RISC processor that executes the MIPS-I instruction set¹. However, the VLSI architecture is quite different from those of MIPS R3000 based RISC processors and processor cores. What this means is that the LX4x80 support MIPS I instructions, their execution and exception models. However, the clocking, pipeline structure, pin-out, and memory interfaces have all been designed to reflect system-on-silicon design needs, deep submicron process technology, as



¹ The MIPS unaligned load and store instructions (LWL, LWR, SWL, SWR) are not supported

well as design methodology advances.

MIPS ISA Execution. The LX4x80 supports the MIPS I programming model. That is, the instruction set incorporates a three-port register file. Two source operands can be supplied and one destination update performed per cycle. The second operand is either a register or 16-bit immediate. The instruction set includes a wide selection of ALU operations executed by the RALU, Lexra's proprietary register based ALU. The RALU also generates memory addresses for 8-bit, 16-bit and 32-bit register loads from (stores to) memory by adding a register base to an immediate offset. Branches are based on comparisons between registers, rather than flags, and are therefore easy to relocate. Optional links following jump or branch instructions assist with subroutine programming.

The MIPS unaligned load and store instructions are not supported, because they represent poor price/performance tradeoff for embedded applications, and because their absence does not affect the software programming model.

Pipeline. LX4180 instructions are executed by a five-stage pipeline and LX4280 instructions are executed in a six-stage pipeline, both of which have been designed so that all transactions internal to the LX4x80 as well as at the interfaces, occur on the positive edge of the processor clock. Unlike most implementations, two-phase clocks are not used. As a result, designs based on the LX4x80 will achieve higher clock speeds in deep sub-micron technologies.

Exception Handling. The MIPS I exception handling model is supported. Exceptions include both instruction-synchronous *traps* as well as hardware and software *interrupts*. The STATUS register controls the interrupt mask and operating mode. Exceptions are prioritized. When an exception is taken, control is transferred to the exception vector, the next instruction address is saved in the EPC register and the exception source is identified in the CAUSE register. A user program located at the exception vector identifies the cause of the exception, and transfers control to the application-specific handler. In the event of an address error exception, the BADVADDR holds the failing address.

Coprocessor Operations. The LX4x80 supports all 32-bit Coprocessor operations. These include moves to and from the Coprocessor general registers and control registers (MTCz, MFCz, CTCz, CFCz), Coprocessor loads and stores (LWCz, SWCz) and branches based on Coprocessor condition flags (BCzT, BCzF). The Lexra-supplied Coprocessor Interface can support all Coprocessor Operations in a single cycle, without pipeline stalls.

1.2 Implementation Overview

The implementation philosophy for the LX4x80 supports the objectives of achieving excellent price/performance and time-to-market. As such, there are two main approaches, which Lexra has taken:

- Deliver either a fully synthesizable Verilog source model or fully implemented hardcore (called SmoothCore™) for popular pure-play foundries.
- Deliver simple building blocks outside the processor core to enable system level customizations such as Coprocessors, application specific instructions, memories, and buses.

The following sections provide an overview of these two points.

1.3 RTL Core & SmoothCore™

Lexra provides two different methods of delivering the LX4x80: RTL Core and SmoothCore. They are aimed respectively at two different development and business models: ASIC & COT.

RTL Core: For ASIC designers, the RTL core is a fully scan-testable Verilog source code of the LX4x80, and can be targeted to any ASIC vendor's standard cell or gate array libraries. In this case, the designer can simply follow the ASIC vendor's design flow to ensure proper sign-off. In addition to the Verilog source code and system level testbench, Lexra provides synthesis scripts as well as floor plan guidelines to maximize the performance of the LX4x80.

Depending upon the particular 0.25μm process, the standard cell library, and the design methodology used, the RTL core should achieve a performance in the range of 100-133 MHz operation. For typical 0.18μm processes the RTL core should achieve 133-166 MHz operation.

SmoothCore: For COT designers who have chips manufactured at popular foundries such as Chartered, TSMC, and UMC, SmoothCore is the quickest, lowest cost, and best performance choice. In this case, the LX4x80 has been fully implemented and verified as a hard macro. All datapath, register file, and interface optimizations have been performed to ensure the smallest die and fastest performance possible. Furthermore, there is a scan based test pattern which provides 99.5% fault coverage during manufacturing tests. SmoothCores achieve 133-166 MHz operation in 0.25μm and 166-200 MHz in 0.18μm processes.

SmoothCore Delivery: Since SmoothCore is delivered as an implemented hard macro, Lexra will provide most of the design database. This includes:

- Encrypted Verilog RTL and gate level models
- Post-layout timing for the target process.
- Behavior model for Motive static timing analysis.
- Layout GDSII
- DRC/LVS result log file

1.4 System Level Building Blocks

The LX4x80 is designed to easily fit into different target applications. It provides the following integration building blocks.

- A simple memory management unit (SMMU).
- An optimized Custom Engine Interface (CEI)
- Up to three Coprocessor Interfaces (CI)
- A flexible Local Memory Interface (LMI) to cache, scratchpad RAM, or ROM.
- A Lexra Bus Controller (LBC) to connect peripheral functions and secondary memories to the processor's own local buses.

The following sections will discuss each of these system building block interfaces.

1.4.1 SMMU

The LX4x80 SMMU is designed for embedded applications using a single address space. Therefore, its primary function is to provide memory protection between user space and kernel space. The SMMU is consistent with the MIPS address space scheme for User/Kernel modes, mapping, and cached/uncached regions. The upper bits of the physical address can be ignored to form a smaller contiguous memory space.

1.4.2 Local Memory Interface

The LX4x80's Harvard Architecture supports two LMI types: instruction memory and a data memory. Both memories can be either asynchronous non-volatile memory (such as ROM, flash, or EPROM), or synchronous RAM. The synchronous memory interface gives designers the fastest interface to memory blocks.

When used as a cache memory controller, the LMI includes a write-through cache mechanism with a two-way set associative instruction cache interface and a direct mapped data cache interface. The tag compare logic as well as a cache replacement algorithm are provided as part of the LMI. One of the instruction cache sets can be locked down as un-swappable local memory.

In all cases, the LMI block is designed to easily interface with standard 32-bit wide memory blocks provided by ASIC vendors or by COT library vendors.

1.4.3 Coprocessor Interface

Lexra supplies an optional Coprocessor Interface (CI) for applications requiring this functionality. Up to three CIs may be implemented in one design. The Coprocessor Interface "eavesdrops" on the Instruction. If a Coprocessor load (LWCz) or "move to" (MTCz, CTCz) is decoded, data will be enabled from the Data Bus into a CI register, then supplied to the designer-defined Coprocessor. Similarly, if a Coprocessor store (SWCz) or "move from" (MFCz, CFCz) is decoded, data will be fetched from the Coprocessor and loaded into a CI register, then transferred onto the Data Bus in the following cycle. The design interface includes a variable-width data bus, five-bit address and independent read and write selects for Coprocessor registers and control registers. The LX4x80 pipeline and Harvard Architecture permit single cycle Coprocessor access and transfer. Designer-defined Coprocessor condition flags CpCondz are synchronized by the CI then passed to the Sequencer for testing in branch instructions.

1.4.4 Custom Engine Interface

The LX4x80 includes a Custom Engine Interface (CEI) which the designer can use to extend the MIPS ALU opcodes with application-specific or proprietary operations. Similar to the standard ALU, the CEI supplies the Custom Engine two input 32-bit operands, SRC1 and SRC2. One operand is selected from the Register File. Depending on the opcode, the second operand is either selected from the Register File or is a 16-bit sign-extended immediate. The opcode is locally decoded by the custom engine and, following execution by the custom engine, the

result is returned on the 32-bit RES bus to the LX4x80. To support slower or multi-cycle operations, a stall signal is supplied by the interface. As an example, the MIPS MULT and DIV instructions are implemented as CEI functions, so designers without the need for multiply or divide do not have to pay the die size penalty.

1.4.5 Lexra Bus Controller

The Lexra Bus Controller (LBC) is the interface between the LX4x80 and the outside world, which includes DRAM and various peripherals. It is a non-multiplexed, non-pipelined, and non-parity checked bus to provide the easiest bus protocol for design integration. On the processor side, the LBC provides a write-buffer of configurable depth to support the write-through cache, as well as the control for byte and half-word transfers. On the peripheral side, LBC is designed to easily interface to industry standard bus protocols, such as PCI, USB and FireWire.

From the silicon implementation standpoint, the LBC can run at any speed from 33 MHz all the way to the speed of the LX4x80 processor core in both the RTL core and SmoothCore. Both cores also support a dedicated full scan interface for the LBC to link to other off-processor devices for DFT purpose.

1.5 Building Block Integration

Several steps have been taken to minimize the time and effort to integrate the LMI, CI, CEI, and LBC.

First, a LX4x80 configuration software tool provides a menu of selections for designers to specify building blocks needed, number of different memory blocks, target speed, target standard cell library, etc. Then the configuration software automatically generates a reference LX4x80 top level Verilog model, makefiles, and scripts for different steps of the supported design flow.

For testability purpose, all building blocks contain scan control signals. These signals can be gathered at the system level, and multiplexed for the DFT software to sequentially test each building block.

1.6 Software Tool Support

LX4x80 programs can be developed with a wide range of third party tools. These tools include the GNU C Compiler and the Green Hills C Compiler. Lexra also supports the Wind River real-time O/S.

1.7 EDA Tool Support

Lexra intends to support all popular EDA software, so designers do not have to alter their design methodology. The following is a snapshot of EDA tools currently supported:

Design Flow	Tools Supported
Simulation	VCS
Synthesis	Design Compiler
Static Timing	Motive
DFT	Test Compiler, TestGen-XP
P&R	Aquarius/XO

Reference Specification

The exact performance of the LX4x80 depends upon the silicon process and the implementation technology used. Furthermore, this specification depends on the memory configuration and CI/CEI selection. The following reference SmoothCore specification for the processor core itself is based upon TSMC's 0.18 μm salicide process.

Specification	LX4180	LX4280
Gate count	75K	150K
Die size (mm ²)	2.5	5.5
Frequency (worst case MHz)	180	200
Power dissipation (mW@1.8V)	90	175
Operating voltage (V)	1.8	1.8
Operating environment	WC Industrial	WC Industrial

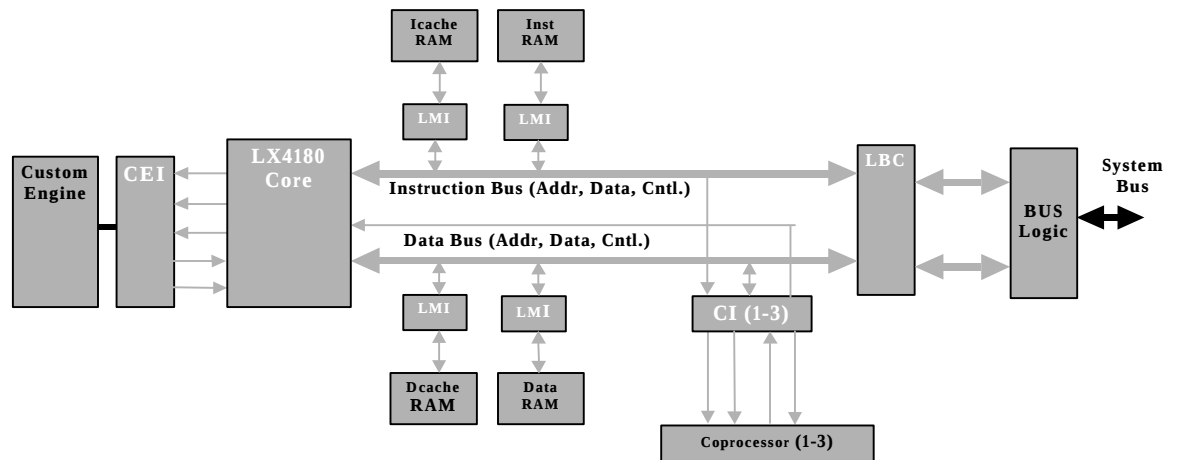
Chapter 2

LX4x80 Overview

MIPS-I ISA Core and System Building Blocks

Figure 1. LX4x80 Block Diagram

The LX4x80 is a processor that executes the MIPS-I instruction set and is targeted



for embedded applications where one or more processors can be integrated with the customer's own ASIC logic. To facilitate ASIC integration and testing, Lexra has developed its proprietary RISC architecture to ensure high performance while maintaining the ability to run existing MIPS-I software. LX4x80 building blocks for on-chip caches, local memories and Lexra system bus controller (LBC) are also included. LX4x80's main applications are embedded control functions for products such as set-up boxes, 56K and xDSL modems, and network protocol processors.

2.1 LX4x80 Key Features

High Performance	Up to 200 MHz for most 0.18 μ processes (LX4280)
Small	Only 2.5 mm ² for the processor core (LX4180)
MIPS-I	Executes MIPS-I instruction set to take advantage of existing software development tools.
ASIC friendly	Single-phase clock to minimize timing complexity. 100% standard cell approach avoids complexity introduced by full custom design.
Portable	Can be ported to any 0.25 μ or 0.18 μ logic and SRAM process.
Customizable	Supports customer-defined Coprocessors and a tightly integrated Custom Engine Interface (CEI) for application-specific user logic.
Building blocks	Interfaces for local Caches or RAM memory. Local instruction caches, data caches and data RAM sizes, up to 16 KB each.
Testability	100% testability using full scan. Low pin out requirement. Optional breakpoint capability accessible by CPU and scan logic.

2.2 Processor Core

Instruction Set. The LX4x80 executes the MIPS-I instruction set with the following exclusions. The MIPS-I unaligned loads and stores (LWL, SWL ,LWR, SWR,) are not supported because they require significant gate count for little benefit in most

applications. The MIPS-I multiply and divide operations (MFHI, MTHI, MFLO, MTLO, MULT, MULTU, DIV, DIVU), which also require significant gate count, are optional opcodes which may be added using the Custom Engine Interface (CEI). The customer therefore can include these functions if the application requires.

The LX4180 incorporates a three-port register file. Two source operands can be supplied and one destination update performed per cycle. The MIPS instruction set includes a wide selection of ALU operations. The second operand is either a register or 16-bit immediate. Data memory addresses are also generated for 8-bit, 16-bit and 32-bit register loads from (stores to) memory by adding a register base to an immediate offset. Branches are based on comparisons between registers, rather than flags, and are therefore easy to relocate. Optional links following jump or branch assist with subroutine programming.

The LX4280 issues dual 32-bit instructions to two distinct 6-stage execution pipelines, Pipe A and Pipe B. The LX4280 incorporates an 8-port (4r/4w) general register file with 4-ports (2r/2w) assigned to Pipe A, and 4-ports (2r/2w) assigned to Pipe B. In each Pipe, one write port is dedicated to register file updates from the Data Bus (Loads, MFCz, CFCz - moves from Coprocessor). The remaining three ports (2r/1w) are available for the other operations assigned to that Pipe. As a result, Loads can dual-issue with any MAC or ALU instruction without register port access restriction.

Pipeline. LX4180 instructions are executed by a five-stage pipeline and LX4280 instructions are executed in a six-stage pipeline, which have been designed so that all transactions internal to the LX4x80 as well as at the interfaces, occur on the positive edge of the processor clock. Unlike most implementations, two-phase clocks are not used. As a result, designs based on the LX4x80 will achieve higher clock speeds in deep sub-micron technologies.

Exceptions. The MIPS R3000 Exception Model is supported. Exceptions include both instruction-synchronous *traps* as well as hardware and software *interrupts*. The STATUS register controls the interrupt mask and operating mode. Exceptions are prioritized. When an exception is taken, control is transferred to the exception vector, the next instruction address is saved in EPC and the exception source is identified in CAUSE. A user program, located at the exception vector, identifies the cause of the exception and transfers control to the application-specific handler.

Coprocessor Operations. The LX4x80 supports all MIPS-I Coprocessor Operations. These include moves to and from the Coprocessor registers (MTCz, MFCz, CTCz, CFCz), Coprocessor loads from data memory (LWCz) and Coprocessor stores to data memory (SWCz).

2.3 System Building Blocks

The LX4x80 processor is designed to easily fit into different target applications. It provides the following building blocks, which are described in this section:

- A Simple Memory Management unit (SMMU).
- A tightly-coupled Custom Engine Interface (CEI)
- A MIPS-I Coprocessor Interface (CI)
- A flexible Local Memory Interface (LMI) that can interface to Caches, EPROM, scratchpad RAM, or ROM.
- The Lexra system bus which implements a PCI-like protocol, allowing customers to integrate their own bus devices with the LX4x80
- The Lexra Bus Controller (LBC), which connects the local buses of the core and local memories to the Lexra system bus.

SMMU. The LX4x80 SMMU is designed for embedded applications using a single virtual address space. Therefore, its primary function is to provide memory protection between user space and kernel space. Nevertheless, it is consistent with the MIPS memory model and programming convention. For applications which use a single unified memory space, the upper bits of the physical address can be ignored to form a smaller contiguous memory space.

Custom Engine Interface. The LX4x80 includes a Custom Engine Interface (CEI) which the customer can use to extend the MIPS ALU opcodes with application-specific or proprietary operations. Similar to the standard ALU, the CEI is supplied two 32-bit input operands, SRC1 and SRC2. One operand is selected from the Register File. Depending on the opcode, the second operand is either selected from the Register File or is a 16-bit sign-extended immediate. The opcode is locally decoded by the Custom Engine and, following execution by the Custom Engine, the result is returned on the 32-bit RES bus to the LX4x80. To support slower or multi-cycle operations, a stall signal is supplied by the interface.

Coprocessor Interface. Lexra supplies an optional Coprocessor Interface (CI) which will be suitable for most applications requiring this functionality. One to three CIs may be implemented in a customer design. The Coprocessor Interface "eavesdrops" on LX4x80 instructions. If a Coprocessor instruction is decoded, the CI moves the data between the customer logic and the LX4x80. The customer

interface includes separate read and write ports, each with five-bit address, read and write selects and data port. The LX4x80 pipeline and Harvard Architecture permit single cycle Coprocessor access and transfer.

Local Memory Interface. The customer can easily configure a variety of memories on the LX4x80 local bus using the Lexra-supplied Local Memory Interface (LMI). Customers never need to directly connect logic to the local busses.

The LX4x80 local memory modules are caches, RAMs and ROMs that service the two local busses, instruction and data, which are connected to the LX4x80 processor core and the system bus via the Lexra Bus Controller (LBC). Customers can optionally integrate one cache, one RAM and one ROM module on each local bus.

Lexra supplies a Local Memory Interface (LMI) for each of the local memory types. The LMIs include the local bus interfaces and logic necessary to implement the function, as well as ports which connect to application-specific RAMs and ROMs. Using a utility program supplied by Lexra, customers configure a top level Verilog module that includes the local bus interconnect and RAM/ROM ports for each Local Memory Interface.

The sizes of the RAMs and ROMs are selectable by the customer. The LX4x80 LMIs use synchronous RAM and ROM modules without parity bits. There is no support for asynchronous memory. (Some ASIC libraries may not support synchronous ROMs. However, this is easily remedied by connecting an address register to the ROM, and connecting the register inputs to the LMI's ROM address port.)

All caches have a 16 byte line size. The valid bits in all cache lines are automatically cleared by the LMIs upon reset. Data caches implement a write-through protocol.

The RAM address sizes accommodate the largest memories that can be configured for a given LMI. When smaller RAMs are used, some high order address bits are not connected. The tag/state RAM widths accommodate the smallest cache that can be configured. When larger caches are configured, some high order tag/state bits are not used.

Lexra System Bus. LX4x80 defines a system bus protocol (the Lexra Bus) which is similar to the Peripheral Component Interface (PCI) bus. Adding new devices to the Lexra Bus is straightforward, achieving high performance with a relatively simple protocol.

Lexra supplies the Lexra Bus Controller (LBC) which connects local busses of the core and local memories to the Lexra system bus. The protocol supports multiple masters, which allows for mastering I/O controllers with DMA engines to be connected to the bus.

The Lexra bus is synchronous. Signals are registered and sampled at the positive edge of the bus clock. The speed of the bus is selectable at 33, 66 and 100MHz.

2.4 Design Integration

To maximize time-to-market and design performance, the LX4x80 is delivered to the licensees as a “hard” core. It is accurately placed and routed using the customer’s target cell library for maximum performance and minimum interconnect, together with timing requirements for timing-driven layout. For verification, the LX4x80 comes with an encrypted RTL model for maximum functional simulation speed, and a detailed gate level model which allows timing back-annotation through the SDF interface.

Manufacturing Test Support. The LX4x80, as well as Lexra-supplied building blocks, are fully scanned. The scan patterns are supplied by Lexra and achieve 99% fault coverage. Of course, customers are responsible for test coverage of customer-designed blocks such as new opcodes executed by the Custom Engine Interface.

Software Tool Support. LX4x80 programs can be developed with a wide range of third party tools. These tools include the GNU C Compiler and the Green Hills C Compiler. Lexra also supports the Wind River real-time O/S.

2.5 Reference Specification

The exact performance of the LX4x80 depends upon the silicon process & the implementation technology used. The following is a SmoothCore reference specification based upon TSMC’s 0.18 μm salicide process:

Specification	LX4180	LX4280
Gate count	75K	150K
Die size (mm ²)	2.5	5.5
Frequency (worst case MHz)	180	200
Power dissipation (mW@1.8V)	90	175
Operating voltage (V)	1.8	1.8
Operating environment	WC Industrial	WC Industrial

2.6 Product Delivery

The LX4x80 is designed to enable cell based IC/ASSP/ASIC designers an easy integration of the processor into the chip. The following design database and guidelines are included as part of the license:

- Synthesizable Verilog RTL model
- Lconfig configuration utility
- Regression tests
- Detailed floorplan and layout guidelines

Chapter

3

The LX4x80 Programming Model

MIPS-I Instruction Set

This section describes the LX4x80 Programming Model. It begins (3.1) with a simplified view of the hardware architecture of the LX4x80 Core, illustrating only those features specifically referenced by the instruction set or user's programs. Next in 3.2, a list summarizing all MIPS-I operations supported by the LX4x80 is presented. These opcodes can be extended by the customer using Lexra's Custom Engine Interface (CEI). This capability is described in section 3.3.

Section 3.4 describes the Simplified Memory Management Unit (SMMU) which is physically incorporated in the LX4x80 LMI. The SMMU provides sufficient memory management capabilities for most embedded applications while insuring execution of third-party MIPS software development tools.

The LX4x80 supports the MIPS R3000 Exception Processing model. The details are described in section 3.5.

The LX4x80 supports all MIPS-I Coprocessor operations. The customer can include one to three application-specific Coprocessors. Lexra provides a functional block called the Coprocessor Interface (CI) which allows the customer a simplified connection between their Coprocessor and the internal signals of the LX4x80. The CI is described in section 3.6.

3.1 Hardware Architecture of the LX4x80 Core

The LX4x80 processor Core (see Figure 2) includes two major blocks: the RALU (Register File and ALU) and Coprocessor 0. The RALU performs ALU operations and generates data addresses while Coprocessor 0 includes instruction address

sequencing and Exception Processing. The RALU and Coprocessor 0 are loosely-coupled and each include their own independent instruction decoders.

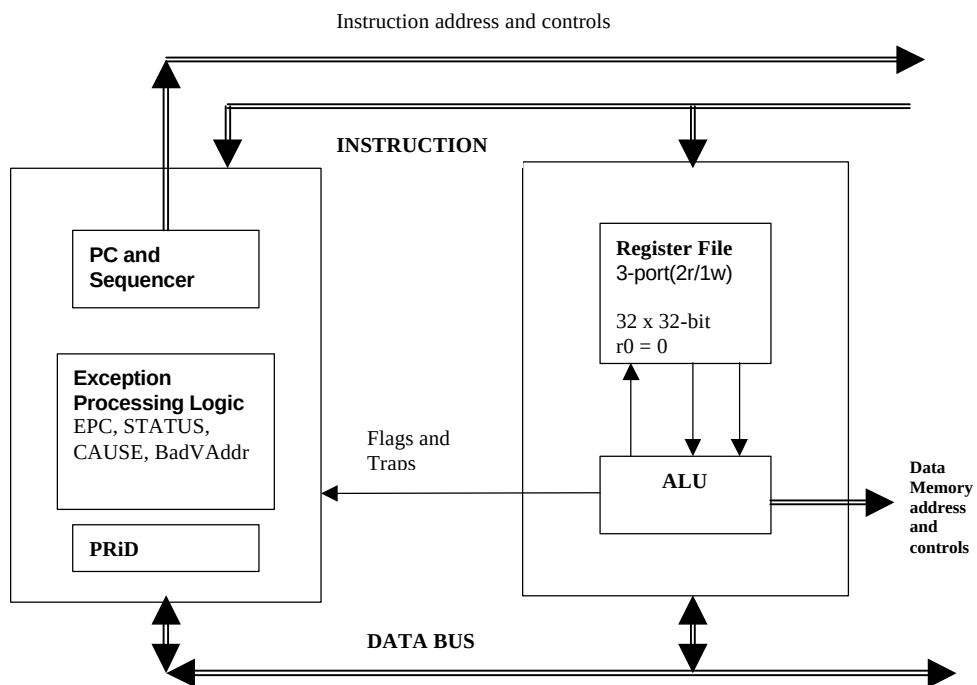


Figure 2. Overview of the LX4x80 Processor Core

3.1.1 LX4180 RALU

The LX4180 RALU incorporates a 32x32b three-port register file. Two source operands can be read and one destination update performed per cycle. The contents of Register 0 is hardwired to 0.

The instruction set includes a wide selection of ALU operations executed by the RALU. In the case of ALU operations, one operand is a register and the second operand is either a register or 16-bit immediate. The immediates are sign-extended or zero-extended, depending on the operation. Two-operand ALU operations are created as special cases of the above by referencing *r0* as one of the sources. Signed adds and subtracts can generate the arithmetic overflow trap *V*, which is sampled by Coprocessor 0.

The RALU also generates the virtual memory addresses for 8-bit, 16-bit and 32-bit register loads from (stores to) memory by adding a register base to a sign-extended 16-bit immediate offset. Data address errors generate the *AdEL*, *AdES* trap flags which are sampled by Coprocessor 0. LX4x80 Data Memory is *Big-Endian*.

Branches are based on comparisons between registers, rather than implicit flags, permitting the programmer more flexibility. As a result of these comparisons, *N* and *Z* flags are generated by the RALU for sampling in Coprocessor 0. The address of the instruction in the memory location following a jump, or branch which is taken, can optionally be stored in one of the general-purpose registers. This register, called the *link*, holds the return address following a subroutine call.

Coprocessor operations permit moves of the general purpose registers to one of three possible application-specific Coprocessors. The general purpose registers can also be loaded from the Coprocessor registers. These transfers occur over the Data Bus similar to data memory loads and stores.

3.1.2 LX4280 Superscalar RALU

To “first approximation” the LX4280 superscalar RALU is a “doubling” of the LX4180 RALU: it includes an 8-port (4r/4w) general register file with 4-ports (2r/2w) assigned to Pipe A, and 4-ports (2r/2w) assigned to Pipe B. In each Pipe, one write port is dedicated to register file updates from the Data Bus (Loads, MFCz, CFCz - moves from Coprocessor). The remaining three ports (2r/1w) are available for the other operations assigned to that Pipe. As a result, Loads can dual-issue with any MAC or ALU instruction without register port access restriction.

Each Pipe has an ALU and a nearly-independent control section. Differences occur in the assignment of operations to Pipe A and Pipe B, and in the pipeline features to support superscalar. The pipeline differences in the RALU to support superscalar issue are:

- Data must be forwarded from Pipe A (Pipe B) to Pipe B (Pipe A) when the input to a Pipe B (Pipe A) execution unit requires a result computed earlier in Pipe A (Pipe B).
- If both Pipe A and Pipe B operations write the same register, the RALU suppresses the write for the earlier instruction based on program order.

The Table lists the detailed assignment of instructions to Pipe A and Pipe B. Pipe B is called the “MAC Pipe” because it uniquely supports multiply-accumulate, as well as multiply and divide operations. The MAC unit, which is optionally attached to Pipe B as Custom Engine 0 (CE0), includes the accumulator registers (including HI and LO) and therefore also supports the *move to* and *move from* operations which transfer data between these registers and the general register file.

Assignment of Instructions of Pipe A, Pipe B

	Pipe A	Pipe B
	The Load / Store Pipe	The MAC Pipe
MIPS 32-bit General Instructions	MIPS 32-bit General Instructions except: CE1 Custom Engine Opcodes, MULT(U), DIV(U), MFHI, MFLO, MTHI, MTLO, MAD(U), MSUB(U)	CE1 Custom Engine Opcodes, MIPS 32-bit ALU Instructions Note: No Load or Store Instructions [MULT(U), DIV(U), MFHI, MFLO, MTHI, MTLO, MAD(U), MSUB(U) – if optional MAC is installed]
MIPS 32-bit Control Instructions	J, JAL, JR, JALR, JALX SYSCALL, BREAK, All Branch Instructions, All COPz, SWCz, LWCz	
MIPS16 Instructions (No Doubleword Instructions)	All MIPS16 Instructions except: MULT(U), DIV(U), MFHI, MFLO	MULT(U), DIV(U), MFHI, MFLO
EJTAG	DERET, SDBBP	

Instructions	(including MIPS16 SDBBP)	
Lexra Control Instructions	MTRU, MFRU, MTRK, MFRK, MTLXC0,MFLXC0	

Pipe A is called the “Load/Store Pipe” because it uniquely supports the Load and Store operations.

The Coprocessor operations, and all “sequencing control instructions” (branches, jumps) are unique to Pipe A. As a result, Pipe B instructions are not routed to Coprocessors.

The opcodes reserved for a customer defined Custom Engine 1 (CE1) are routed to Pipe B, since CE1 is attached to Pipe B.

3.1.3 Coprocessor 0

Coprocessor 0 is responsible for instruction address sequencing and for exception processing.

For normal execution, the next instruction address has several potential sources: the increment of the previous address, a branch address computed using a pc-relative offset, or a jump target address. For jump addresses, the absolute target can be included in the instruction, or it can be the contents of a general-purpose register transferred from the RALU.

The LX4x80 implements branch-prediction, so that in the event of prediction failure, a stall cycle is incurred and the correct address is selected from a special “backup” register. Statistics from several large programs suggest that these stalls will degrade average LX4x80 throughput by approximately 5%. However, the net effect of the LX4x80’s branch prediction on performance is positive because this technique eliminates certain critical paths and therefore, permits a higher speed system clock.

If an *exception* occurs, one of several hardwired vectors is selected for the next instruction address. The exception vector depends on the mode and specific trap which occurred. This is described further in the section below on *Exception Processing*.

The following registers, which are visible to the programming model, are located in Coprocessor 0:

Coprocessor 0 register	Function
EPC	Holds address for return after exception handler.
PRiD	Processor ID (read-only)
STATUS	Interrupt masks, mode selects
CAUSE	Exception cause
BadVAddr	Holds bad virtual address if address exception error occurs.
CCTL	Cache control

Table 1. Coprocessor 0 Registers

EPC, STATUS, CAUSE and BadVAddr are described further in the section on Exception Processing below. PRiD is a read-only register which allows the customer's software to identify the specific version of the LX4x80 which has been implemented in their product. The CCTL register is a Lexra defined Coprocessor0 register used to control the instruction and data caches, described in Chapter 5.

The contents of the above registers can be transferred to and from the RALU's general-purpose register file using Coprocessor 0 operations. (Unlike registers located in Coprocessors 1-3 they cannot be loaded or stored directly to data memory.)

3.1.4 The LX4180 Instruction Pipeline

Instructions are executed by a five-stage pipeline, which has been designed so that all transactions internal to the LX4180 as well as at the interfaces, occur on the positive edge of the processor clock. Unlike most implementations, two-phase clocks are not used. As a result, designs based on the LX4180 will achieve higher clock speeds in deep sub-micron technologies.

LX4180 Pipeline:

Stage 1 I	Instruction fetch
Stage 2 S	Source fetch
Stage 3 E	Execution
Stage 4 M	Memory data select
Stage 5 W	Write back to register file

Forwarding paths are used in the LX4180 so that the five-stage pipeline is not visible to the programming model. In other words, the results of an instruction (with few exceptions) can be used as source in the next instruction.

3.1.5 The LX4280 Instruction Pipeline

The LX4280 issues dual 32-bit instructions to two distinct 6-stage execution pipelines, Pipe A and Pipe B. *Superscalar* architectures have been widely deployed in RISC CPUs where increased performance is obtained at the cost of increased area.

Compared to the conventional five stage pipeline employed by the LX4180, the LX4280 adds one additional stage, the D (Decode) stage.

LX4280 Pipeline:

Stage 1 I	Instruction fetch
Stage 2 D	Decode
Stage 3 S	Source fetch
Stage 4 E	Execution
Stage 5 M	Memory data select
Stage 6 W	Write back to register file

The LX4280 I-Cache and IRAM can fetch two 32-bit instructions I0_I, I1_I simultaneously. Following the superscalar instruction buffer and issue logic, described below, the instructions issue as IB_S_R to Pipe B and IA_S_R to Pipe A. To avoid degrading performance, the superscalar issue logic operates during the Decode-Stage of the pipeline which occurs after I (Instruction Fetch) and before S (Source Fetch). The D stage allows the system clock cycle time to be not only maintained, but improved, compared to the five stage pipeline. Support for fully synchronous memories in the LX4280 has the added benefit of isolating the processor logic from the customer-supplied memories, thus facilitating integration of the LX4280 into SoC designs.

As a result of the D-Stage, a two cycle penalty is incurred on branch prediction failure vs. the one-cycle penalty in the LX4180 five stage pipeline.

3.2 Summary of LX4x80 Instructions

The LX4x80 executes MIPS-I instructions as detailed in the tables below. To summarize, the LX4x80 executes MIPS-I instructions with the following exclusions: the unaligned loads and stores (LWL, SWL, LWR, SWR) are not supported

because they add significant silicon area for little benefit in most applications. Occasional unaligned data handling is more cost effectively handled without these specialized ops. The MIPS-I fixed point multiply and divide operations (MFHI, MTHI, MFLO, MTLO, MULT, MULTU, DIV, DIVU) also require significant silicon area which would be wasted in many applications. The customer can optionally add these functions using the Custom Engine Interface (CEI) described below. The unaligned loads and stores as well as the multiply and divide instructions (unless implemented using the CEI) cause a Reserved Instruction trap.

3.2.1 ALU Instructions

Instruction	Description
ADD rD, rA, rB	$rD \leftarrow rA + \{rB, \text{immediate}\}$
ADDU rD, rA, rB	Add reg rA to either reg rB or a sign-extended 16-bit immediate. Result is stored in reg rD. ADD and ADDI can generate overflow trap; ADDU and ADDIU do not.
ADDI rD, rA, immediate	
ADDIU rD, rA, immediate	
SUB rD, rA, rB	$rD \leftarrow rA - rB$
SUBU rD, rA, rB	Subtract reg rB from reg rA. Result is stored in register rD. SUB can generate overflow trap. SUBU does not.
AND rD, rA, rB	$rD \leftarrow rA \& \{rB, \text{immediate}\}$
ANDI rD, rA, immediate	Logical <i>and</i> of reg rA with either reg rB or a zero-extended 16-bit immediate. Result is stored in reg rD.
OR rD, rA, rB	$rD \leftarrow rA \mid \{rB, \text{immediate}\}$
ORI rD, rA, immediate	Logical <i>or</i> of reg rA with either reg rB or a zero-extended 16-bit immediate. Result is stored in reg rD.
XOR rD, rA, rB	$rD \leftarrow rA \wedge \{rB, \text{immediate}\}$

XORI rD, rA, immediate	Logical <i>xor</i> of reg rA with either reg rB or a zero-extended 16-bit immediate. Result is stored in reg rD.
NOR rD, rA, rB	$rD \leftarrow \sim(rA \mid rB)$ Logical <i>nor</i> of reg rA with either reg rB or a zero-extended 16-bit immediate. Result is stored in reg rD.
LUI rD, immediate	$rD \leftarrow \text{immediate} \parallel 16'(0)$ The 16-bit immediate is stored into the upper half of reg rD. The lower half is loaded with zeroes.
SLL rD, rB, immediate SLLV rD, rB, rA	$rD \leftarrow rB \ll \{rA, \text{immediate}\}$ Reg rB is left-shifted by 0-31. The shift amount is either the 5b immediate or the 5 lsb of rA. Result is store in reg rD.
SRL rD, rB, immediate SRLV rD, rB, rA	$rD \leftarrow rB \gg \{rA, \text{immediate}\}$ Reg rB is right-shifted by 0-31. The unsigned shift amount is either the 5b immediate or the 5 lsb of rA. Result is stored in reg rD.
SRA rD, rB, immediate SRAV rD, rB, rA	$rD \leftarrow rB \gg(a) \{rA, \text{immediate}\}$ Reg rB is arithmetic right-shifted by 0-31. The unsigned shift amount is either the 5b immediate or the 5 lsb of rA. Result is stored in reg rD.
SLT rD, rA, rB SLTU rD, rA, rB SLTI rD, rA, immediate SLTIU rD, rA, immediate	$rD \leftarrow 31'(0) \parallel 1 \text{ if } rA < \{rB, \text{immediate}\} \text{ else } 0$ If reg rA is less than {rB, immediate} set rD to 1, else 0. The 16-bit immediate is sign extended. For SLT, SLTI, the comparison is signed; for SLU, SLTIU, the comparison is unsigned.

--	--

3.2.2 Load and Store Instructions

Instruction	Description
LB rD, immediate(rA)	$rD \leftarrow \text{Memory}[rA + \text{immediate}]$
LBU rD, immediate(rA)	Reg rD is loaded from data memory. The memory address is computed as <i>base + offset</i>, where the base is reg rA and the offset is the sign-extended 16-bit immediate. LB, LBU addresses are interpreted as byte addresses to data memory; LH, LHU as halfword (16-bit) addresses; LW as word (32-bit) addresses. The data fetched in LB, LH (LBU, LHU) is sign-extended (zero-extended) to 32-bits for storage to reg rD. rD cannot be referenced in the instruction following a load instruction.
LH rD, immediate(rA)	
LHU rD, immediate(rA)	
LW rD, immediate(rA)	
SB rB, immediate(rA)	$rB \rightarrow \text{Memory}[rA + \text{immediate}]$
SH rB, immediate(rA)	Reg rB is stored to data memory. The memory address is computed as <i>base + offset</i>, where the base is reg rA and the offset is the sign-extended 16-bit immediate. SB addresses are interpreted as byte addresses to data memory; the 8 lsb of rB are stored. SH addresses are interpreted as halfword addresses to data memory; the 16 lsb of rB are stored.
SW rB, immediate(rA)	

3.2.3 Branch and Jump Instructions

Instruction	Description
BEQ rA, rB, destination BNE rA, rB, destination	$pc \leftarrow (pc + 4) + 14'(\text{dest}(15)) \parallel \text{dest} \parallel 00$ if COND else $pc \leftarrow (pc + 8)$ where COND = (rA = rB) for EQ, (rA ne rB) for NE For BEQ, BNE the instruction after the branch (<i>delay slot</i>) is always executed.
BLEZ rA, destination BGTZ rA, destination	$pc \leftarrow (pc + 4) + 14'(\text{dest}(15)) \parallel \text{dest} \parallel 00$ if COND else $pc \leftarrow (pc + 8)$ where COND = (rA <= 0) for LE, (rA > 0) for GT For BLEZ, BGTZ the instruction after the branch (<i>delay slot</i>) is always executed.
BLTZ rA, destination BGEZ rA, destination	$pc \leftarrow (pc + 4) + 14'(\text{dest}(15)) \parallel \text{dest} \parallel 00$ if COND else $pc \leftarrow (pc + 8)$ where COND = (rA < 0) for LE, (rA >= 0) for GT For BLTZ, BGEZ the instruction after the branch (<i>delay slot</i>) is always executed.
BLTZAL rA, destination BGEZAL rA, destination	Same as the BLTZ and BGEZ except that the address of the instruction following the delay slot is saved in r31 (regardless of whether the branch is taken.)
J target	$pc \leftarrow pc(31:28) \parallel \text{target} \parallel 00$ target is a 26-bit absolute. The instruction following J (<i>delay slot</i>) is always executed.

JAL target	Same as above except that the address of the instruction following the delay slot is saved in r31.
JR rA	$pc \leftarrow (rA)$ The instruction following JR (delay slot) is always executed.
JALR rA, rD	Same as above except that the address of the instruction following the delay slot is saved in rD.

3.2.4 Control Instructions

Instruction	Description
SYSCALL	The Sys Trap occurs if SYSCALL is executed.
BREAK	The Bp Trap occurs if BREAK is executed.
RFE	Causes the KU/IE stack to be popped. Used when returning from the exception handler. See "Exception Processing" below.
SLEEP	Initiates low-power standby mode. This is a Lexra specific operation (LEXOP). See section on "Power Savings Mode".

3.2.5 Coprocessor Instructions

Instruction	Description
LWCz rCGEN, immediate(rA)	$rCGEN \leftarrow \text{Memory}[rA + \text{immediate}]$ Coprocesor z general reg rCGEN is loaded from data memory. The memory address is computed as <i>base + offset</i> , where the base is reg rA and

	the offset is the sign-extended 16-bit immediate. rCGEN cannot be referenced in the following instruction.
SWCz rCGEN, immediate(rA)	$rCGEN \rightarrow \text{Memory}[rA + \text{immediate}]$ Coprocessor z general reg rCGEN is stored to data memory. The memory address is computed as <i>base + offset</i>, where the base is reg rA and the offset is the sign-extended 16-bit immediate.
MTCz rB, rCGEN CTCz rB, rCCON	In MTCz(CTCz) the general register rB is moved to Coprocessor z general (control) reg rCGEN(rCCON). rCGEN and rCCON cannot be referenced in the following instruction.
MFCz rB, rCGEN CFCz rB, rCCON	In MFCz(CFCz) the Coprocessor z general (control) reg rCGEN(rCCON) is moved to the general register rB. rB cannot be referenced in the following instruction.
BCzT destination BCzF destination	$pc \leftarrow (pc + 4) + 14'(\text{dest}(15)) \parallel \text{dest} \parallel 00$ if COND else $pc \leftarrow (pc + 8)$ where COND = (CpCondz = True) for BCzT, (CpCondz = False) for BCzF. For BCzT, BCzF the instruction after the branch (<i>delay slot</i>) is always executed.
COPz	INST[24:0] are transferred to Coprocessor z for decode and execution.

3.3 Opcode Extension Using the Custom Engine Interface (CEI)

3.3.1 CEI Operations

Customers can add proprietary or application-specific opcodes to their LX4x80 based products using the Custom Engine Interface (CEI). The new instructions take one of the following forms illustrated below and use reserved opcodes.

New Instruction	Description	Available Opcodes
NEWOPI rD, rA, immediate	$rD \leftarrow rA \text{ NEWOPI immediate}$ Reg rA is supplied to the SRC1 port of CEI and the 16-bit immediate, sign-extended to 32-bits is supplied to SRC2. The result of the customer's NEWOPI is placed on the CEI input port RES and stored in reg rD.	INST[31:26] = 24 - 27
NEWOPR rD, rA, rB	$rD \leftarrow rA \text{ NEWOPR } rB$ Reg rA is supplied to the SRC1 port of CEI and reg rB is supplied to SRC2. The result of the customer's NEWOPI is placed on the CEI input port RES and stored in reg rD.	INST[31:26] = 0 and INST[5:0] = 56,58-60,62-63

Lexra permits customer operations to be added using the eight (8) I-Format opcodes and eight (8) R-Format opcodes listed in the Table above. Integer multiply/divide operations (MULT, MULTU, DIV, DIVU, MTLO, MTHI, MFLO, MFHI) will be supplied by Lexra and will use the standard MIPS-I opcodes for these

operation when they are (optionally) included in the CEI. Other opcode extensions in future Lexra products will *not* utilize the opcodes reserved above for proprietary customer extensions.

When the CEI decodes NEWOPI or NEWOPR, it must signal the Core that a custom operation has been executed so that the Reserved Instruction trap will not be taken. Multi-cycle custom operations can be executed by asserting CESEL.

Note: the custom operation may choose to ignore the SRC1 and SRC2 operands supplied by the Core and reference customer registers instead. Results can also be written to an implicit customer register; however, unless $rD = 0$ is coded, a register in the Core will also be written.

3.3.2 Interface Signals

Signal	Type	Description
SRC1[31:0]	OUTPUT	Operand supplied to customer logic.
SRC2[31:0]	OUTPUT	Operand supplied to customer logic.
RES[31:0]	INPUT	Result of customer logic. Supplied to Core.
CEIOP[11:0]	OUTPUT	Instruction OP and SUBOP fields – to be decoded by customer logic.
CEHALT	INPUT	Indicates that a multi-cycle custom operation is in progress.
CESEL	INPUT	Indicates that a CEI operation has been decoded.

3.4 Memory Management

The LX4x80 includes a Simplified Memory Management Unit (SMMU) for the instruction address and the data memory address. These units are physically located in the Local Memory Interface (LMI) modules. The hardwired virtual-to-physical address mapping performed by the SMMU is sufficient to insure execution of third-party software development tools.

Virtual Address Space	Description	Mapped to Physical Address
0xC000_0000 to 0xFFFF_FFFF	KSEG2. 1Gbyte. Addressable in kernel mode. Cached.	0xC000_0000 to 0xFFFF_FFFF
0xA000_0000 to 0xBFFF_FFFF	KSEG1. 0.5 Gbyte. Addressable only in kernel mode. Uncached. Used for I/O devices.	0x0000_0000 to 0x1FFF_FFFF
0x8000_0000 to 0x9FFF_FFFF	KSEG0. 0.5 Gbyte. Addressable only in kernel mode. Cached.	0x0000_0000 to 0x1FFF_FFFF (differentiated from KSEG1 addresses with an internal signal)
0x0000_0000 to 0x7FFF_FFFF	KUSEG. 2Gbyte. Addressable in kernel or user mode. Cached.	0x4000_0000 to 0xBFFF_FFFF

Note: the 0.5 Gbyte of physical address space from 0x2000_0000 to 0x3FFF_FFFF is not accessible with the above memory map.

3.5 Exception Processing

The LX4x80 implements the MIPS R3000 exception processing model as described below. Features specific to on-chip TLB support are not included. In the discussion below, the term *exception* refers to both *traps*, which are non-maskable program synchronous events, and *interrupts*, which result from unmasked asynchronous events. This section first lists the exceptions supported by the LX4x80 and their associated priorities. Next the LX4x80 registers STATUS,

CAUSE, EPC and BadVaddr involved in exception processing are listed. Finally, exception processing entry and exit is described.

3.5.1 Exception List

The list below is numbered from highest to lowest priority. ExcCode is stored in CAUSE when an exception is taken. Note that Sys, Bp, RI, CpU can share the same priority level because only one can occur in a particular time slot.

Exception	Priority	ExcCode	Description
Reset	1	--	Reset trap.
AdEL – instruction	2	4	Address exception trap. Instruction fetch. Occurs if the instruction address is not word-aligned or if a kernel address is generated in user mode.
Ov	3	12	Arithmetic overflow trap. Can occur as a result of signed add or subtract operations.
Sys	4	8	SYSCALL instruction trap. Occurs when SYSCALL instruction is executed.
Bp	4	9	BREAK instruction trap. Occurs when BREAK instruction is executed.
RI	4	10	Reserved instruction trap. Occurs when a reserved opcode is fetched. Reserved opcodes are listed below.
CpU	4	11	Coprocessor Usability trap. Occurs when an attempt is made to execute a Coprocessor n operation and Coprocessor n is not enabled.

AdEL – data	5	4	Address exception trap. Data fetch. Occurs if the data address is not properly aligned or if a kernel address is generated in user mode.
AdES	6	5	Address exception trap. Data store. Occurs if the data address is not properly aligned or if a kernel address is generated in user mode.
Int	7	0	Unmasked interrupt. There are six (6) level-sensitive hardware interrupt request signals into the LX4x80 Core. Each is synchronized by the Core to the LX4x80 system clock. In addition, program writes to CAUSE[9:8] are software-initiated interrupt requests. Each of the eight (8) requests has an associated mask bit in STATUS. Int is generated by any unmasked request (when Interrupts are globally enabled).

3.5.2 Exception Processing Registers: STATUS, CAUSE, EPC, BadVAddr

STATUS. Coprocessor 0 General Register Address = 12

31-28	27-23	22	21-16	15-8	7-6	5	4	3	2	1	0
CU(3:0)	0	BEV	0	IM(7:0)	0	KUo	IEo	KUp	IEp	KUc	IEc

CU	CU[n] = 1(0) indicates that Coprocessor n is usable(unusable) in Coprocessor instructions.
BEV	Bootstrap Exception Vector. Selects between two trap vectors. (see below)
IM	Interrupt masks for the six hardware interrupts and two software interrupts.
KU/IE	KU = 0(1) indicates kernel(user) mode. In the LX4x80 user mode virtual addresses must have msb = 0. In kernel mode, the full address space is addressable. IE = 1(0) indicates that interrupts are enabled(disabled). KUo IEo KUp IEp KUc IEc forms a three-level stack hardware stack KU/IE signals. The <i>current</i> values are KUc/IEc, the <i>previous</i> values are KUp/IEp, and the <i>old</i> values (those before previous) are KUo/IEo. (see below)
STATUS	is read or written using MTC0 and MTF0 operations. On reset, BEV = 1, KUc = IEc = 0. The other bits in STATUS are undefined. The 0 fields are ignored on write and are 0 on read. It is recommended that the user explicitly write them to 0 to insure compatibility with future versions of the LX4x80.

■ CAUSE. Coprocessor 0 General Register Address = 13

31	30	29-28	27-16	15-8	7	6-2	1-0
BD	0	CE(1:0)	0	IP(7:0)	0	ExcCode(4:0)	0

BD	Branch Delay. Indicates that the exception was taken in a branch delay slot.
CE	Coprocessor Exception. In the case of a Coprocessor Usability exception, indicates the number of the responsible Coprocessor.
IP	Interrupt Pending. Each bit in IP(7:0) indicated an associated unmasked interrupt request.
ExcCode	The ExcCode listed above for the different exceptions are stored here when an exception occurs.
CAUSE	is read or written using MTC0 and MTF0 operations. The only program writable bits in CAUSE are IP(1:0), which are called <i>software interrupts</i> . CAUSE is undefined at reset. The 0 fields are ignored on write and are 0 on read.

■ **EPC. Coprocessor 0 General Register Address = 14**

EPC is a 32-bit read-only register which contains the virtual address of the next instruction to be executed following return from the exception handler. If the exception occurs in the delay slot of a branch, EPC will hold the address of the branch instruction and BD will be set in CAUSE. The branch will typically be re-executed following the exception handler.

■ **BadVAddr. Coprocessor 0 General Register Address = 8**

BadVAddr is a 32-bit read-only register which holds the virtual address (instruction or data) which generated an AdEL or AdES exception error.

3.5.3 Exception Processing: Entry and Exit

When an exception occurs, the instruction address changes to one of the following locations:

RESET	0xbfc0_0000
Other exceptions, BEV = 0	0x8000_0080
Other exceptions, BEV = 1	0xbfc0_0180

The KU/IE stack is pushed:

KUo | IEo | KUp | IEp | KUc | IEc →(push)

KUp | IEp | KUc | IEc | 0 | 0

which disables interrupts and puts the program in kernel mode. The code (ExcCode) for the exception source is loaded into CAUSE so that the application-specific exception handler can determine the appropriate action. The exception handler should not re-enable Interrupts until necessary context has been saved.

To return from the exception handler, program execution should be returned to the address stored in EPC. To do so, EPC must first be moved to a general register using MFC0 then followed by a JR operation. RFE only *pops* the KU/IE stack:

KUp | IEp | KUc | IEc | 0 | 0 →(pop)

KUp | IEp | KUp | IEp | KUc | IEc

(This example assumes that KU/IE were not modified by the exception handler). Therefore, a typical sequence of operations to return from the exception handler would be:

MFC0 EPC, r26 // r26 is a temporary storage register in the RALU

Etc.

JR r26

RFE

3.6 The Coprocessor Interface (CI)

The customer can implement up to three Coprocessors to interface with the LX4x80. The contents of these Coprocessors can include up to thirty-two (32) 32-bit *general registers* and up to thirty-two (32) 32-bit *control registers*. The general registers can be moved to and from the RALU's registers using MTCz, MFCz operations. They can also be loaded and stored from data memory using LWCz, SWCz operations. The control registers can only be moved to and from the RALU's registers using CTCz, CFCz operations.

Lexra supplies a simple Coprocessor Interface (CI) model so that the customer can easily interface a Coprocessor to the LX4x80. The CI supplies a set of address

control, address, and data busses which can be tied directly to the Coprocessor general and special registers.

Signal	Type	Description
CzRDADDR(4:0)	OUTPUT	Read address to a Coprocessor z control or general register.
CzRDGEN	OUTPUT	Indicates read of Coprocessor z general register.
CzRDCON	OUTPUT	Indicates read of Coprocessor z control register.
CzRDDATA(31:0)	INPUT	Read data from Coprocessor z control or general register.
CzWRADDR(4:0)	OUTPUT	Write address to a Coprocessor z control or general register.
CzWRGEN	OUTPUT	Indicates write to Coprocessor z general register.
CzWRCON	OUTPUT	Indicates write to Coprocessor z control register.
CzWRDATA(31:0)	OUTPUT	Write data to Coprocessor z control or general register.

The addresses, output data and control signals are supplied to the user's Coprocessor on the rising edge of SYSCLK. In the case of a read cycle, the Coprocessor must supply the data from either the control or general register on CzRDDATA(31:0) by the end of the same cycle. Similarly, the write of data from CzWRDATA(31:0) to the addressed control or general register must be complete by the end of the cycle.

The CI incorporates a forwarding path so that data which is written in Instruction(N) can be read in Instruction(N + 2). The Coprocessor registers should be implemented as positive-edge flip-flops using the LX4x80 system clock.

3.7 Power Savings Mode

The operating system kernel can initiate a power savings standby mode using the Lexra specific SLEEP instruction. This holds the LX4x80's internal clocks in the high state until an external hardware interrupt is received.

Before executing the SLEEP instruction, the kernel ensures that the interrupt condition that will ultimately terminate standby mode has been enabled via the IM field of the coprocessor 0 Status register. When the SLEEP instruction enters the W stage, the standby logic will stall the processor and wait for any outstanding processor initiated memory writes to complete. After these are completed, the standby logic holds the system bus clocks high. These are held high until an enabled interrupt is received.

When standby mode is terminated by an interrupt, the standby logic allows the clocks to toggle. The processor honors the interrupt by branching to the exception handler as is normally done for interrupt servicing. Because several instructions are held in the pipeline while the clocks are frozen prior to the interrupt, the exception PC will not point to the SLEEP instruction, but rather some instructions later. Typically, a kernel would enter an idle loop just after executing the SLEEP instruction, so the interrupt will be serviced from the context of the kernel's normal idle interrupt service level.

The LX4x80 will take a minimum of 6 cycles after the SLEEP instruction enters the W stage to safely synchronize the initiation of standby mode, i.e. hold the clocks in the high state. Two cycles are required to terminate standby mode. The processor is stalled during these periods.

The standby logic receives the free running system and bus clocks, and generates gated clocks for distribution to the LX4x80. The standby logic must use flip-flops tied to free running clocks, which results in about a dozen loads on the free running clocks. Applications that do not require standby mode may bypass the standby logic to reduce clock distribution complexity.

Two pins, SL_SLEEPING_R and SL_SLEEPING_BR, are available from the standby logic and are asserted high when the processor is in standby mode. The _R pin is for use in the system clock domain, and the _BR pin is for use in the bus clock domain.

Chapter

4

MIPS16

Instruction Extensions for Code Density

MIPS16 is an Application Specific Extension (ASE) to the MIPS Instruction Set Architecture (ISA) which was developed to improve code density, especially for System-on-Chip (SoC) designs. In these designs, on-chip instruction storage is often a significant, even dominant, portion of the silicon component cost. This is especially true for real-time applications because, in order to meet real-time requirements, instruction cache miss penalties cannot be tolerated and thus a large portion of the instruction storage must be resident on-chip.

MIPS16 provides a set of 16-bit instruction formats to encode the most common operations. The key compromises required to achieve 16-bit encoding include: (i) some MIPS I instructions are not available, (ii) immediate widths are reduced, (iii) only 8 of the 32 general registers may be directly addressed. As a result some operations cannot be executed in MIPS16 or require multiple MIPS16 instructions. Thus realistic programs need to include both MIPS16 and MIPS I instructions, using MIPS16 where possible to save storage, at some cost to performance.¹ Mode switching between MIPS16 and MIPS I is discussed below. In order to permit occasional access to all 32 general registers without the overhead of mode switching, MIPS16 provides *MOVE* instructions to move data between the MIPS16-visible registers and the full general register set. Also, to permit occasional use of 16-bit immediates without mode switching, MIPS16 provides the *EXTEND*

¹ The MIPS16 performance penalty results from occasionally using two instructions where one MIPS I instruction would suffice. Some of this penalty is recovered in applications where a larger number of instructions per cache line reduces cache miss rate.

instruction to allow a full width immediate in two MIPS16 instruction cycles. (Programs requiring a large register set or frequent full-width immediates should be compiled in MIPS I.)

MIPS16 is difficult to program effectively at the assembler level. This is because of the limited register set and the restricted size immediates. In fact, according to Sweetman¹, "MIPS16 is not a suitable language for assembly coding. Rather, MIPS16 is viewed as a compiler option which can be effectively applied to achieve significant code size reduction where performance is not critical."²

4.1 MIPS16 Instructions

This section describes the MIPS16 instructions, with emphasis on the differences between MIPS16 and the 32-bit MIPS ISA. The first table lists MIPS I Instructions that are *not supported* in MIPS16.

The second table lists MIPS I instructions *which are supported* in MIPS16. In most cases these are specialized versions of the MIPS I instruction. MIPS16 is compatible with MIPS I, II and III, IV or V. The LX4x80 implements *all* MIPS16 for 32-bit data operations.³ The table lists all MIPS16 instructions together with the corresponding MIPS I instruction and the specific specialization required to produce the MIPS16 instruction (other than smaller register set and smaller immediates).

The third table lists the several new instructions introduced by MIPS16.

It is notable that MULT(U), DIV(U) are supported in MIPS16 unlike ARM's Thumb mode. MFHI and MFLO are also supported and are necessary to access the result of MULT(U) or DIV(U). However, MTHI and MTLO are not supported. These are used primarily to restore the state after exception handling and are used within the kernel, typically in MIPS I.

¹ "See MIPS Run", Dominic Sweetman, Appendix D, p. 425.

² Benchmarks on MIPS16 code compression have been reported by Microprocessor Report. [reference]

³ MIPS16 includes 16-bit formats for a number of MIPS III 64-bit doubleword operations which are not supported either in the MIPS I ISA or in the LX4x80.

Unsupported MIPS I Instructions	Assembler Mnemonics
Coprocessor operations	CTCz, CFCz, MTCz, MFCz, LWCz, SWCz, BCzT, BCzF, COPz
Unaligned loads, stores	LWL, LWR, SWL, SWR
Arithmetic operations	ADD, ADDI, SUB
Conditional branches	BLEZ, BGTZ, BLTZ, BGEZ, BLTZAL, BGEZAL
Logical operations with immediates	ANDI, ORI, XORI, LUI
Jumps	J
Miscellaneous	SYSCALL, RFE, MTHI, MTLO

MIPS16 Instructions (which are supported in the 32-bit MIPS ISA)	Specialized version of 32-bit MIPS instruction for MIPS16 ¹
LB(U) ry, offset(rx) LH(U) ry, offset(rx) LW ry, offset(rx) LW rx, offset(sp) (r29 base) SB ry, offset(rx) SH ry, offset(rx) SW ry, offset(rx) SW rx, offset(sp) (r29 base)	LW rx, offset(base); base = r29 SW rx, offset(base); base = r29
ADDIU ry, rx, immediate ADDIU rx, immediate ADDIU sp, immediate (1-operand)	ADDIU rt, rs, immediate; rt=rs ADDIU rt, rs, immediate; rt=rs=r29

¹If no 32-bit MIPS instruction is listed, no specialization beyond limited size register set and limited size immediates is required.

ADDIU rx, sp, immediate (2-operand)	ADDIU rt, rs, immediate; rs=r29
ADDU rz, rx, ry	
SUBU rz, rx, ry	
NEG rx, ry (2-operand)	SUBU rd, rs, rt; rs=r0
SLT(U) rx, ry (r24 dest. implied)	SLT(U) rd, rs, rt; rd=r24
SLTI(U) rx, immediate (2-op., r24 dest)	SLTI(U) rt, rs, immediate; rt=rs
CMPI rx, immediate (r24 dest. implied)	XORI rt, rs, immediate; rt=r24
CMP rx, ry (r24 dest. implied)	XOR rd, rs, rt; rd=r24
AND rx, ry (2-operand)	AND rd, rs, rt; rd=rs
OR rx, ry (2-operand)	OR rd, rs, rt; rd=rs
XOR rx, ry (2-operand)	XOR rd, rs, rt; rd=rs
NOT rx, ry (2-operand)	NOR rt, rs, rt; rs=r0
MOVE ry, r32 (2-operand)	OR rd, rs, rt; rs=r0
MOVE r32, ry (2-operand)	OR rd, rs, rt; rs=r0
LI rx, immediate	ORI rd, rs, immediate; rs=r0
SLL rx, ry, immediate	
SRL rx, ry, immediate	
SRA rx, ry, immediate	
SLLV ry, rx (2-operand)	SLLV rd, rt, rs; rd=rs
SRLV ry, rx (2-operand)	SRLV rd, rt, rs; rd=rs
SRAV ry, rx (2-operand)	SRAV rd, rt, rs; rd=rs
MULT(U) rx, ry	
DIV(U) rx, ry	
MFHI rx	
MFLO rx	
JAL target	
JR rx	
JR ra	JR rs; rs=r31

JALR ra, rx (2-operand; link = r31)	JALR rs, rd; rs=r31
BEQZ rx, offset (1-operand)	BEQ rs, rt, offset; rt=r0
BNEZ rx, offset (1-operand)	BNE rs, rt, offset; rt=r0
BTEQ offset (implied operands)	BEQ rs, rt, offset; rs=r24, rt=r0
BTNE offset (implied operands)	BNE rs, rt, offset; rs=r24, rt=r0
B offset (implied operands)	BEQ rs, rt, offset; rs=r0, rt=r0
BREAK	

As noted earlier, MIPS16 restricts the MIPS I directly addressable register set and immediate field. Another common MIPS16 restriction is that two, rather than three, register operands, are permitted. MIPS16 provides a number of assembler mnemonics which are unfamiliar from MIPS I but are really just special cases of MIPS I which were important enough to provide in MIPS16 but where the more general case could not be fully encoded.

Instructions New to MIPS16 (not available in MIPS 32-bit ISA)	Comment
LW rx, offset(pc)	Load word with pc-relative address
ADDIU rx, pc, immediate	ADDIU with pc operand
EXTEND immediate	Supplies 11-bit immediate for use in the following MIPS16 instruction
JALX target	Jump to target, store return in r31 <i>and toggle the ISA mode between MIPS16 and MIPS I.</i>

The pc-relative load LW is important to overcoming the drawback of smaller immediates in MIPS16. It allows full 32-bit immediates to be embedded in the program and loaded into registers in a single instruction. The ADDIU with pc operand is useful to support immediates embedded in the program. The pc value referenced in LW or ADDIU depends on the context of the p-relative instruction as shown in the table:

Context for pc-Relative Instruction	pc Value
Normal case. (Non-extended pc-relative instruction, not in jump delay slot.)	pc of the pc-relative instruction.
pc-relative instruction with extended immediate	pc of the EXTEND instruction
Non-extended pc-relative in the delay slot of jump, JR, JALR, JAL(X) (extended instructions are not permitted in the delay slot of the jump.)	pc of the jump instruction

EXTEND is used to supply an extra 11-bits of immediate. It is used together with the restricted size immediate field of the next instruction to supply a full width immediate. EXTEND cannot occur in the delay slot of a Jump. It is not necessary for the assembly programmer to code EXTEND instructions. It will automatically be assembled by MIPS16 assemblers wherever the immediate is too large to be encoded in a single MIPS16 instruction.

Another new instruction JALX, is available in both MIPS16 and also in MIPS I on machines implementing MIPS16 and is discussed below. [in MIPS I machines not implementing MIPS16, the JALX opcode 000111 causes an RI trap.]

4.2 Mode switching

Mode is switched between MIPS16 and MIPS I in one of two ways:

1. The instruction,

JALX target

toggles the mode.

2. The lsb of the general register rx in

JR rx
JALR rs, rx (in MIPS16 rs=ra)

causes the mode to be set to MIPS16 if rx[0] = 1; to MIPS I if rx[0] = 0. However, the lsb of the instruction memory address from JR/JALR is forced to 0. As a consequence, machines which implement MIPS16 never take AdEL exceptions on the lsb of the instruction address (This is true regardless of whether the machine is operating in MIPS16 or MIPS I mode.).

The mode bit is saved in the lsb of the link register in JAL, JALX, JALR.

4.3 Exceptions

Upon Exception, the mode is automatically switched to MIPS I. The mode is saved in the lsb of the Exception PC (EPC). EPC[0] = 0 indicates that the Exception occurred while executing code in MIPS I mode; EPC[0] = 1 indicates that the Exception occurred in MIPS16 mode. The typical program will save the EPC to a general register and later return to the main program with a JR instruction, causing the proper ISA mode to be restored.

4.4 No Delay Slots

Consistent with the MIPS16 emphasis on code density, there are no load delay or branch delay slots. In other words, the instruction following the branch is executed only if the branch is not taken. [MIPS16 *jumps* (JAL, JALX, JR, JALR) have a single delay slot, the same as in MIPS I. For jumps, the target address is always taken. Thus, there is no risk that the delay slot cannot be used to do useful work: the instruction from the target can be moved to the delay slot, if necessary.]

For MIPS16 loads, the instruction following the load can reference the loaded register (as in MIPS II). This is because the MIPS I compiler is not always successful in scheduling a useful instruction in the delay slot and must occasionally resort to a NOP, reducing code density. This possibility is eliminated in MIPS16.

4.5 Effect on System Clock

The MIPS16 developers intended for the MIPS16 instructions to be "mapped" to the MIPS I instructions, then executed. However, this approach while reducing power and area by a small amount, has the drawback of appreciably lengthening the critical path in the Instruction Fetch (IF) pipeline stage and thus compromising the system clock speed on products implementing MIPS16, even for MIPS I code execution. This is evident in some product specs which give one system clock spec when MIPS16 is used and one when not used (NB: it is not practical to switch clock speed with mode, thus *any* application of MIPS16 in these products must occur at the lower speed.) Lexra, by careful use of dual decoders, avoids the mapping technique and *offers MIPS16 without reducing the MIPS I system clock.*

Chapter

5

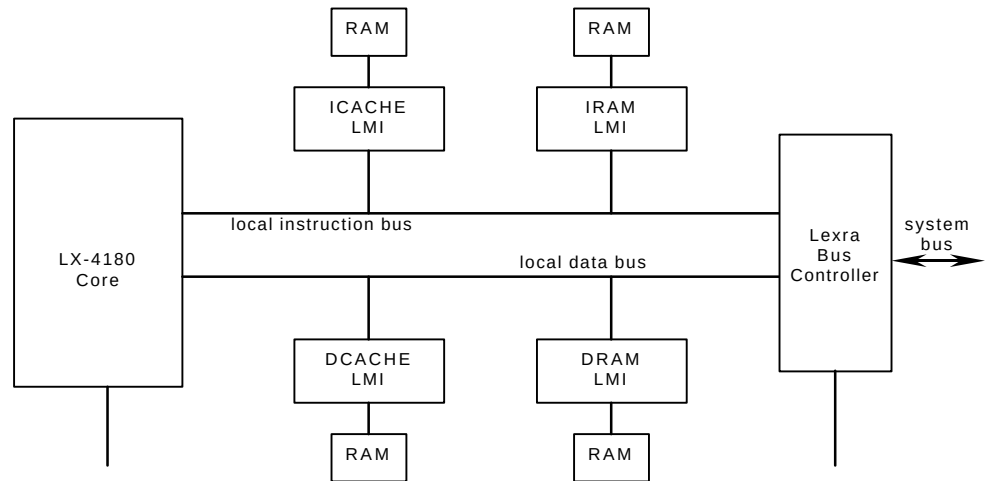
Local Memory

Caches, RAMs and ROMs

This chapter describes the memories which customers may configure on the LX4x80 local bus. First, a brief summary of the local bus conventions and available modules is given (this section). Next, each of the local memory interfaces supplied by Lexra is described, starting with section 5.1.

This material discusses configuration options, and the ports that customers must access to connect application specific RAM and ROM devices that are used by the LX4x80 local memory interfaces. All of the signals between the processor core, the LMIs and the system bus controller are automatically configured by the LX4x80 configuration tool. Customers never directly connect logic to the local bus itself, so only a limited amount of information about the local bus is presented here.

The LX4x80 local memory modules are caches and RAMs that service the two local busses, instruction and data, which are connected to the LX4x80 processor core and system bus via the Lexra Bus Controller (LBC). One cache must be integrated on each local bus. One RAM module is optional on each local bus. The top-level structure with emphasis on the local buses is shown in the figure below.



Lexra supplies a Local Memory Interface (LMI) for each of the local memory types. The LMIs include the local bus interfaces and logic necessary to implement the function, as well as ports which connect to application specific RAMs and ROMs. Using the Iconfig utility supplied by Lexra, customers configure the Verilog code that includes the local bus interconnect and memory ports for each Local Memory Interface in their application.

Customers attach elements such as RAMs and ROMs to each LMI's application specific interfaces, documented in the sections that follow. The sizes of the RAMs and ROMs are selectable by the customer. The LX4x80 LMIs directly support synchronous RAMs, whereby the address, write data and write enable are registered at the RAM inputs. ROMs may also be connected, but may require a customer supplied address register at the address inputs.

Lexra supplies an integration layer for the LMIs and the memory devices connected to them. In this layer, memory devices are instantiated as generic modules satisfying the depth and width requirements for each specific memory instance. The Iconfig utility supplies a summary of the memory devices required for the chosen configuration. In most cases, customers simply need to write a wrapper that connects the generic module port list to a technology specific RAM instance inside the RAM wrapper.

All caches have a 16 byte line size. The valid bits in all cache lines are automatically cleared by the LMIs upon reset. Data caches implement a write-through protocol. Caches do not snoop the system bus.

The RAM address sizes shown in the port lists in the following sections accommodate the largest memories that can be configured for a given LMI. When smaller RAMs are used, some high order address bits are not used (drop one high order INDEX bit per factor of two cache size). The tag/state RAM widths accommodate the smallest cache that can be configured. When larger caches are configured, some low order tag/state bits are not used (drop one low order bit per factor of two cache size).

The table below summarizes the LMIs that can be integrated on the local busses.

Name	Description
ICACHE	Direct mapped or two-way set associative instruction cache.
IRAM	Instruction RAM or ROM.
DCACHE	Direct mapped data cache.
DRAM	Data RAM or ROM.

Table 2. Local Memory Interface Modules

5.1 Cache Control Register: CCTL

CCTL. Coprocessor 0 General Register Address = 20

31-4	3-2	1	0
Reserved	ILock	IInval	DInval

When reading this register, the contents of the Reserved field are undefined. When writing this register, the contents of the Reserved field should be preserved.

The ILock field controls set locking in the two set associative instruction cache. When ILock is 00 or 01, the instruction cache operates normally. When ILock is 10, all cached instruction references are forced to occupy set 1. The hardware will invalidate lines in set 0 if necessary to accomplish this. When ILock is 11, lines in set 1 are never displaced – i.e. they are locked in the cache. Set 0 is used to hold other lines as needed.

To utilize the cache locking feature, software should execute at least one pass of critical subroutines or loops with ILock set to 10. After this has been done, ILock should be set to 11 to lock the critical code into set 1, and use set 0 for other code.

The IInval and DInval fields control hardware invalidation of the instruction cache, instruction RAM and data cache. A 0 to 1 transition on IInval will initiate a hardware invalidation sequence of the entire instruction cache, and if it is present, the instruction RAM. Likewise, a 0 to 1 transition on DInval will initiate a hardware invalidation sequence of the entire data cache. The data RAM, if present, is unaffected by this operation.

The hardware invalidation sequence for the instruction and data caches requires one cycle per cache line to complete. The invalidation sequence for the instruction RAM requires one cycle per 32 lines to complete.

Depending on the circumstances, software may be able to employ an alternative to a full invalidation of the data cache. If a small number of lines must be invalidated, software may perform cached reads from aliases of memory locations of concern. This will displace whatever data is in the addressed locations of the data cache, even if they do not cache the affected memory location.

Another alternative, if the affected memory location has an alias in uncacheable (Kseg1) space, is to simply perform an uncached read of the affected memory locations. If the location is resident in the data cache it will be invalidated. This method has the advantage of not displacing data in the cache unless it is absolutely necessary to maintain coherency.

With either of these two alternatives, it is only necessary to reference one word of each affected cache line.

5.2 Instruction Cache (ICACHE) LMI

The ICACHE LMI supplies the interface for a direct mapped or two-way set associative instruction cache attached to the LX4x80 local bus. The degree of associativity is specified through Iconfig. The ICACHE LMI participates in cacheable instruction fetches, but only if the address is not claimed by the IRAM module. The configurations supported by ICACHE, and the synchronous RAMs required for each, are summarized in the table below. The inclusion of the ICACHE LMI on the local bus is not optional – it is required for proper LX4x80 operation.

Note that the instruction store for the two-way ICACHE consists of two 32-bit wide banks, with separate write-enable controls. The tag store consists of one RAM

bank with tag and valid bits for the two set members, and an extra bit used by the LRU replacement algorithm. However, all of these tag store elements are controlled by a single write-enable signal. The cache controller automatically performs a read-modify-write cycle of the tag RAM to accomplish updates to a subset of the tag RAM fields.

Configuration	ICACHE_INST RAM	ICACHE_TAG RAM
1,024 bytes, 2-way	2 x 128 x 32 bits	32 x 49 bits
2,048 bytes, 2-way	2 x 256 x 32 bits	64 x 47 bits
4,096 bytes, 2-way	2 x 512 x 32 bits	128 x 45 bits
8,192 bytes, 2-way	2 x 1,024 x 32 bits	256 x 43 bits
16,384 bytes, 2-way	2 x 2,048 x 32 bits	512 x 41 bits
1,024 bytes, direct mapped	256 x 32 bits	64 x 23 bits
2,048 bytes, direct mapped	512 x 32 bits	128 x 22 bits
4,096 bytes, direct mapped	1,024 x 32 bits	256 x 21 bits
8,192 bytes, direct mapped	2,048 x 32 bits	512 x 20 bits
16,384 bytes, direct mapped	4,096 x 32 bits	1,024 x 19 bits

Table 3. ICACHE Configurations

The table below lists the ICACHE signals that are connected to application specific modules. The IC_ prefix indicates signals that are driven by the ICACHE LMI module and received by the RAMs. The ICR_ prefix indicates signals that are driven by the ICACHE RAMs and received by the ICACHE LMI. Lexra supplies the Verilog module that makes all required connections to these wires.

Signal	Description
IC_TAGINDEX[1:4]	Tag and state RAM address (line).
ICR_TAGRD0[23:0]	Tag and state RAM element 0 read path.
ICR_TAGWR0[23:0]	Tag and state RAM element 0 write path.
ICR_TAGRD1[23:0]	Tag and state RAM element 1 read path.
IC_TAGWR1[23:0]	Tag and state RAM element 1 write path.
ICR_TAGLRURD	Tag and state RAM LRU bit read path
IC_TAGLRUWR	Tag and state RAM LRU bit write path
IC_TAGWE	Tag and state RAM write enable.
IC_INSTINDEX[13:2]	Instruction RAM address (word).
ICR_INST0RD[31:0]	Instruction RAM element 0 read path.
ICR_INST1RD[31:0]	Instruction RAM element 1 read path
IC_INSTWR[31:0]	Instruction RAM write path (to both elements).
IC_INST0WE	Instruction RAM element 0 write enable.
IC_INST1WE	Instruction RAM element 1 write enable.

Table 4. ICACHE RAM Interfaces**5.3 Fill-Once Instruction RAM (IRAM) LMI**

The IRAM LMI supplies the interface for a fill-once instruction store. The IRAM behaves like an instruction cache, except that it serves a fixed part of the address space that is the same size as its instruction store. Thus, the IRAM cache lines require valid bits, but no tags. Once a line has been filled in the IRAM, it remains valid until it is cleared through explicit software intervention via the CCTL register described in section 5.1.

The IRAM module participates in any cacheable or uncacheable instruction fetch that falls within its configured range, as described below. The IRAM is a convenient, low-cost alternative method that makes the contents of an external ROM or memory region available to the core for high-speed access.

The configurations supported by IRAM, and the synchronous RAMs required for each, are summarized in the table below.

Configuration	IRAM_INST RAM	IRAM_STATE RAM
1,024 bytes, direct mapped	256 x 32 bits	2 x 32 bits
2,048 bytes, direct mapped	512 x 32 bits	4 x 32 bits
4,096 bytes, direct mapped	1,024 x 32 bits	8 x 32 bits
8,192 bytes, direct mapped	2,048 x 32 bits	16 x 32 bits
16,384 bytes, direct mapped	4,096 x 32 bits	32 x 32 bits
32,768 bytes, direct mapped	8,192 x 32 bits	64 x 32 bits
65,536 bytes, direct mapped	16,384 x 32 bits	128 x 32 bits

Table 5. IRAM Configurations

The table below lists the IRAM signals that are connected to application specific modules. The IW_ prefix indicates signals that are driven by the IRAM LMI module and received by RAMs. The IWR_ prefix indicates signals that are driven by RAMs and received by the IRAM LMI. The IWC_ prefix identifies configuration ports on the IRAM LMI that are typically wired to constant values. Lexra supplies the Verilog module that makes all required connections to these wires.

IWC_ wires define where the IRAM is mapped into the physical address space. This configuration information defines the local bus address region of the IRAM. It also determines the address of the external resources which are accessed when an IRAM miss occurs. The Iconfig utility supplied by Lexra will verify that the configured address range does not interfere with other regions defined by the

LX4x80. Note that the actual size of the memory element need not be a power of two. The configuration allows any size with line granularity.

To use the IRAM LMI as a ROM controller, the IWR_VALIDRD lines can be tied to logic high, thus forcing hits.

Signal	Description
IW_INSTINDEX[15:2]	Decoded IRAM instruction address (word).
IWR_INSTRD[31:0]	Instruction read data.
IW_INSTWR[31:0]	Instruction write data.
IW_INSTWE	Instruction RAM write enable
IW_VALIDINDEX[15:9]	Decoded IRAM valid flag offset (32 flags)
IWR_VALIDRD	Valid bit read.
IW_VALIDWR	Valid bit write
IW_VALIDWE	Valid bit RAM write enable
IWC_BASE[31:16]	Configured base address (modulo 64K bytes).
IWC_TOP[15:4]	Configured top address (modulo 16 bytes).

Table 6. IRAM RAM Interfaces

5.4 Direct Mapped Write Through Data Cache (DCACHE) LMI

The DCACHE LMI supplies the interface for a direct mapped write through data cache attached to the LX4x80 local bus. The DCACHE LMI participates in cacheable data reads and writes, but only if the address is not claimed by the DRAM LMI. The configurations supported by DCACHE, and the synchronous RAMs required for each, are summarized in the table below. The inclusion of the DCACHE LMI on the local bus is not optional – it is required for proper LX4x80 operation.

The direct mapped DCACHE module services word read requests from the core in one cycle when the request hits the cache. Byte or half-word reads that hit the data cache require an extra cycle for alignment. The data cache stream word writes that hit the cache at the rate of one per cycle. Byte or half-word writes that hit the cache require an extra cycle to merge the data with the current line contents. When a cache write operation is immediately followed by a cache read, the cache must delay the read for one cycle while the write completes.

When a miss occurs, the cache obtains a cache line (four words) of data from the Lexra Bus Controller (LBC). Write operations that hit the data cache are simultaneously written into the cache and forwarded to the write buffer of the LBC. Thus, if the core subsequently reads the data, it will likely be available from the

cache. Data writes that miss the cache are forwarded to the write buffer of the LBC without disturbing any data currently in the cache.

Configuration	DCACHE_DATA RAM	DCACHE_TAG RAM
1,024 bytes, direct mapped	256 x 32 bits	64 x 23 bits
2,048 bytes, direct mapped	512 x 32 bits	128 x 22 bits
4,096 bytes, direct mapped	1,024 x 32 bits	256 x 21 bits
8,192 bytes, direct mapped	2,048 x 32 bits	512 x 20 bits
16,384 bytes, direct mapped	4,096 x 32 bits	1,024 x 19 bits

Table 7. DCACHE Configurations

The table below lists the DCACHE signals that are connected to application specific modules. The DC_ prefix indicates signals that are driven by the DCACHE LMI module and received by the RAMs. The DCR_ prefix indicates signals that are driven by the DCACHE RAMs and received by the CCACHE LMI. Lexra supplies the Verilog module that makes all required connections to these wires.

Signal	Description
DC_TAGINDEX[9:0]	Tag and state RAM address (line).
DCR_TAGRD[23:0]	Tag and state RAM read path.
DC_TAGWR[23:0]	Tag and state RAM write path.
DC_TAG_WE	Tag and state RAM write enable.
DC_DATAINDEX[11:0]	Data RAM address (word).
DCR_DATARD[31:0]	Data RAM read path.
DC_DATAWR[31:0]	Data RAM write path.
DC_DATAWE	Data RAM write enable.

Table 8. DCACHE RAM Interfaces

5.5 Scratch Pad Data RAM (DRAM) LMI

The DRAM LMI supplies the interface for a scratch pad data RAM attached to the LX4x80 local bus. The DRAM module participates in any cacheable or uncacheable data read or write operation that falls within its configured range, as described below. Read/write performance is identical to that of data cache hits, described in the previous section.

The configurations supported by DRAM, and the synchronous RAMs required for each, are summarized in the table below.

Configuration	DRAM_DATA RAM
1,024 bytes	256 x 32 bits
2,048 bytes	512 x 32 bits
4,096 bytes	1,024 x 32 bits
8,192 bytes	2,048 x 32 bits
16,384 bytes	4,096 x 32 bits
32,768 bytes	8,192 x 32 bits
65,536 bytes	16,384 x 32 bits

Table 9. DRAM Configurations

The table below lists the DRAM signals that are connected to application specific modules. The DW_ prefix indicates signals that are driven by the DRAM LMI module and received by RAMs. The DWR_ prefix indicates signals that are driven by RAMs and received by the DRAM LMI. The DWC_ prefix identifies configuration ports on the DRAM LMI that are typically wired to constant values. Lexra supplies the Verilog module that makes all required connections to these wires.

The DWC_ wires define where the DRAM is mapped into the physical address space. This configuration information defines the local bus address region of the IRAM. It is not possible for any DRAM reference to result in an operation on the system bus. The Iconfig utility supplied by Lexra will verify that the configured address range does not interfere with other regions defined by the LX4x80.. Note that the actual size of the memory element need not be a power of two. The configuration allows any size with line granularity.

The DRAM LMI may also be used as a ROM controller.

Signal	Description
DW_DATAINDEX[14:0]	Decoded data RAM index (word).
DWR_DATARD[31:0]	Data RAM read data.
DWR_DATAWR[31:0]	Data RAM write data.
DWR_DATAWE	Data RAM write enable.
DWC_BASE[31:16]	Configured base address (modulo 64K bytes).
DWC_TOP[15:4]	Configured top address (modulo 16 bytes).

Table 10. DRAM RAM Interfaces

Chapter

6

The Lexra System Bus

Please see file Lexra_System_Bus.PDF in the documentation directory.

The Lexra System Bus is ...

Chapter

7

EJTAG

Debugging hooks in the LX4x80

The host computer communicates to the EJTAG probe through either a serial or parallel port or ethernet connection. The probe, in turn, communicates to the LX4x80 EJTAG hardware via the included IEEE 1149.1 JTAG interface. Through the use of the JTAG TAP controller, probe data is shifted in to the EJTAG data and control registers to respond to processor requests, DMA into system memory, configure the EJTAG control logic, enable single-step mode, or configure the EJTAG breakpoint registers. Through the use of the EJTAG control registers, the user can set hardware breakpoints on the instruction cache address, data cache address or data, and LBUS address and data.

Currently, Embedded Performance Inc. (EPI) and Green Hills Inc. provide EJTAG debuggers and probes for the LX-4180.

The LX4x80 EJTAG implements version 1.5.3 of the EJTAG specification, and includes support for the following features:

- Processor access of host via addressing of probe memory space.
- Host probe can DMA directly into system memory or I/O devices.
- Hardware breakpoints on internal cache busses
- Single-step execution mode
- Real-time Program Counter Trace
- New Debug exception and 2 new instructions, including an instruction that raises the debug exception via software.

Chapter

8

Customizable Interfaces

Customer Access Points to the LX4x80

The LX4x80 provides customer access points for the Lexra system bus, Custom Engine, and Coprocessor Interfaces. This section provides a detailed signal list for these access points. Attachment of memory devices to the LMIs is not considered in this section. Those are described in Chapter 5.

Lexra supplies the top-level RTL code that integrates customer logic with the signals described here. In the tables that follow, “input” refers to an input to the Lexra core logic, and “output” refers to an output from Lexra core logic.

System Signals		
input	SYSCLK	Processor core clock
input	BUSCLK	System bus clock
input	RESET_D1_R_N	Reset delayed one clock
input	RESET_D1_BR_N	Bus clock domain reset delayed one clock
input	IntreqN[5:0]	Interrupt request lines
input	LWBDISABLE	LBC write buffer disable control 1 = buffer disabled, 0 = buffer enabled
input	LBCSYNCMODE	LBC bus interface mode 1 = synchronous mode, 0 = asynchronous
input	MEMSEQUENTIAL	Declares main memory line order policy 1 = sequential, 0 = interleave
input	MEMZEROFIRST	Declares main memory first word policy 1 zero first, 0 - desired word first
input	MEMFULLWORD	Declares main memory partial write 0 = partial writes, 1 = no partial writes

Custom Engine Signals		
input	CE1_RES_E[31:0]	Result from CE
output	CE1_CE1HOLD	Hold signal to CE
input	CE1_SEL_E_R	CE op decoded
input	CE1_HALT_E_R	Pipeline stall term
output	CE1_CE1OP_S_R[11:0]	CE opcode
output	CE1_INSTM32_S_R_N	0 = 16-bit instruction mode; 1 = 32-bit
output	CE1_AOP_E_R[31:0]	A operand
output	CE1_BOP_E_R[31:0]	B operand
output	Exception	Exception signal from processor core
Coprocessor Z (1-3)		
input	Cond_inZ	Cop flag
output	CZrd_addr[4:0]	Cop read address
output	CZrhold	Hold for address register
output	CZrd_gen	General register read command
output	CZrd_con	Control register read command
output	CZwr_addr[4:0]	Register write address
output	CZwr_gen	General register write command
output	CZwr_con	Control register write command
output	CZwr_data[31:0]	Register write data
input	CZrd_data[31:0]	Register read data
LBC System Bus Interface		
output	LAddrO[31:0]	LBC address
output	LDataO[31:0]	LBC data
input	LDatal[31:0]	System data
output	Llrdy	LBC initiator ready
input	Llrdyl	System initiator ready
output	LFrame	LBC transaction frame
input	LFrameI	System transaction frame
input	LSel	System slave select
input	LTrdy	System target ready
output	LCmd[6:0]	LBus command
output	LReq	LBC bus request
input	LGnt	System bus grant
output	LCoe[¹ LBC_COE-1:0]	LBC command output enable terms
output	LXoe[¹ LBC_XOE-1:0]	LBC transaction output enable terms
output	LDoe[¹ LBC_DOE-1:0]	LBC data output enable terms

9

Multiply-Divide-Accumulate

The Optional MAC-DIV Module

The Multiply/Divide/Accumulate (MAC-DIV) module is an optional feature of the LX4x80 family of embedded microprocessors. This chapter discusses the operation and features of the MAC-DIV module.

9.1 Summary of Features

Mnemonic	Operation	Latency	Repeat Delay	Description
MTHI	HI <- Rs	- ^(*)	-	pre-load accumulator, or restore saved HI
MTLO	LO <- Rs	-	-	pre-load accumulator, or restore saved LO
MFHI	Rd <- HI	1	-	read accumulator, or part of 64 bit result
MFLO	Rd <- LO	1	-	read accumulator, or part of 64 bit result
MULT	{HI,LO} <- Rs * Rt	5	-	32x32 signed multiply 64bit result
MULTU	{HI,LO} <- Rs * Rt	5	-	32x32 unsigned multiply, 64bit result
MADH	HI <- HI + Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, with 32 bit signed add to accum
MADL	LO <- LO + Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, with 32 bit signed add to accum
MAZH	HI <- 0 + Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, add to pre-zeroed 32bit accum
MAZL	LO <- 0 + Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, add to pre-zeroed 32bit accum
MSBH	HI <- HI - Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, with 32 bit signed sub from accum
MSBL	LO <- LO - Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, with 32 bit signed sub from accum

MSZH	HI <- 0 - Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, sub from pre-zeroed 32bit accum
MSZL	LO <- 0 - Rs[15:0] * Rt[15:0]	3	0	16x16 signed multiply, sub from pre-zeroed 32bit accum
DIV	HI <- Rs%Rt ; LO <- Rs/Rt	35	-	32 by 32 signed divide with remainder
DIVU	HI <- Rs%Rt ; LO <- Rs/Rt	35	-	32 by 32 unsigned divide with remainder

(*) A dash ("-") means that the quantity is not meaningful to the instruction.

9.2 High-Lights

- MAZH, MAZL, MSZH, MSZL are like MADH etc. but pre-zero the accumulator, with no performance penalty. They can also be used for lower latency MULT if operands are 16 bits each.
- All ops are supported in M16 mode as well as M32 for best code compression.
- Independent 32-bit HI and LO accumulators for Multiply-accumulate. Allows more overlap of operations while waiting for completion of results.
- Multiply-subtract instructions eliminate need to negate coefficients.
- Use of HI/LO registers as target of instructions eliminates stalls while operation completes, allowing other instructions to be executed in parallel without tricky score boarding techniques.
- In case of resource conflicts, hardware manages all hazards simplifying software debug.
- There are no coding restrictions.

9.3 Op-codes for standard mode (32-bit) instructions

Mnemonic	31 - 26	25 - 21	20 - 16	15 - 6	5 - 0
MFHI	000000	rs	rt	0000000000	010000
MTHI	000000	rs	rt	0000000000	010001
MFLO	000000	rs	rt	0000000000	010010
MTLO	000000	rs	rt	0000000000	010011

MULT	000000	rs	rt	0000000000	011000
MULTU	000000	rs	rt	0000000000	011001
DIV	000000	rs	rt	0000000000	011010
DIVU	000000	rs	rt	0000000000	011011
MADH	111100	rs	rt	0000000000	000000
MADL	111100	rs	rt	0000000000	000010
MAZH	111100	rs	rt	0000000000	000100
MAZL	111100	rs	rt	0000000000	000110
MSBH	111100	rs	rt	0000000000	010000
MSBL	111100	rs	rt	0000000000	010010
MSZH	111100	rs	rt	0000000000	010100
MSZL	111100	rs	rt	0000000000	010110

9.4 Op-codes for MIPS-16 (16 bit) mode instructions:

Mnemonic	15 - 11	10 - 8	7 - 5	4 - 0
MFHI	11101	rx	ry	10000
MTHI	not supported by MIPS-16 architecture			
MFLO	11101	rx	ry	10010
MTLO	not supported by MIPS-16 architecture			
MULT	11101	rx	ry	11000
MULTU	11101	rx	ry	11001
DIV	11101	rx	ry	11010
DIVU	11101	rx	ry	11011
MADH	11111	rx	ry	00000
MADL	11111	rx	ry	00010
MAZH	11111	rx	ry	00100
MAZL	11111	rx	ry	00110
MSBH	11111	rx	ry	10000
MSBL	11111	rx	ry	10010
MSZH	11111	rx	ry	10100
MSZL	11111	rx	ry	10110

Notes:

The op-codes are unchanged (from the MIPS-I standard) for the existing MULT, DIV, MF, and MT instructions. In M32 mode, the new instructions are all R-format with bits 31:26 = 6'b111100. Bits 5:0 determine the specific operation, as shown. In M16 mode, the new instructions are all RR-format with bits 15:11 = 5'b111111. Bits 4:0 determine the specific operation, as shown.

For the MAxx, MSxx instructions, the upper 16 bits of both operand registers are ignored.

The MxxH and MxxL instructions can be freely interleaved. That is, adds and subtracts from either accumulator can be combined in a sequence with the two accumulators functioning "in parallel."

The MxZx instructions can be used as standalone 16-bit signed multiply. An additional intent is the need for a "MTHI, zero" instruction at the beginning of a multiply-accumulate sequence, for example:

```
MAZH r1,r2
MADH r3,r4
MADH r5,r6
MADH r7,r8
any op that doesn't write HI
any op that doesn't write HI
MFHI r9
```

In the above sequence, the two non-HI ops are not necessary for correct operation but the pipeline will stall if they are not used, so it is more efficient to perform useful work in those slots.

For the MULTx instructions, the most efficient use is:

```
MULTx r1,r2
any op that doesn't write HI or LO
any op that doesn't write HI or LO
any op that doesn't write HI or LO
any op that doesn't write HI or LO
MFLO r3      /* LO is available 1 cycle before HI */
MFHI r4
```

9.5 Multiplier Pipelining

A single multiplier supports all of the multiply instructions.

The MAD, MAZ, MSB, MSZ instructions, which have 16-bit operands are implemented in a pipelined fashion, with single cycle throughput and 3 cycle latency.

The MSxx instructions are implemented by negating the multiplier for the 16-bit multiplication but are otherwise identical to the corresponding MAXx instructions. This has the effect of subtracting the product of the original operands from the accumulator.

The MULT instructions, which have 32-bit operands use the same hardware in an iterative fashion to generate the 64 bit result, with 5 cycle latency for the low order 32 result bits. The high order 32 result bits are available 1 cycle later.

The HI and LO registers are used as two independent 32-bit accumulators for the 16 bit multiply instructions or as a paired 64-bit result for the 32-bit multiply instructions.

Note: There is no indication of overflow for the 32-bit add portion of the 16-bit multiply-accumulate instructions. The MFHI(LO) instruction will stall the pipeline until the results of the most recent instruction which stores into HI(LO) has completed.

9.6 Accessing HI and LO after multiply instructions

The MFLO(HI) instruction reads the contents of the LO(HI) register during the E cycle of the pipeline. The following descriptions indicate how the latency of the multiply instructions affects the usage of the MF instructions. The most efficient sequence is shown. If the MF instruction is coded earlier, the correct result will still be obtained because the hardware will stall the MF instruction in the E-cycle until the result is valid.

During the E cycle of any multiply operation, the initial operands are re-coded and loaded into the MANDHW and MIERHW (MBOOTH) registers. For the MULTx operations, the multiply cycles can be labeled M1 through M5. Then the following timing diagram is valid:

MULTx	I	S	E	M1	M2	M3	M4	M5											
any op		I	S	E	M	W													
any op			I	S	E	M	W												
any op				I	S	E	M	W											
any op					I	S	E	M	W										
MFLO						I	S	E	M	W									
MFHI							I	S	E	M	W								
LO valid									X	X									
HI valid										X									

For the MADx operations, the pipeline cycles after E can be labeled as C (carry save), and A (accumulate). Then the following timing diagram is valid:

MAZH0	I	S	E	C	A						
MADH1		I	S	E	C	A					
MADH2			I	S	E	C	A				
MADH3				I	S	E	C	A			
any op					I	S	E	M	W		
any op						I	S	E	M	W	
MFHI							I	S	E	M	W
HI contains						A0	A1	A2	A3		

9.7 Divider Overview and Register Usage

Given a dividend DEND, and divisor DVSR, the divider generates a quotient QUOT and remainder REM that satisfy the following conditions, regardless of the signs of DEND and DVSR:

$$\begin{aligned} \text{DEND} &= \text{DVSR} * \text{QUOT} + \text{REM}, \\ 0 &\leq \text{abs}(\text{REM}) < \text{abs}(\text{DVSR}) \end{aligned}$$

where REM and DEND have the same sign.

It is worth noting that the requirement that REM and DEND have the same sign is not universally accepted if DEND and DVSR are not both positive. (For example the Modula-3 language expects: $-5\text{DIV}3=-2$, $-5\text{MOD}3=+1$, whereas the divider generates $\text{QUOT}=-1$, $\text{REM}=-2$ in agreement with FORTRAN and others.) These examples show the possible combinations of signs:

DEND	DVSR	QUOT	REM
----	----	----	---
+19	+5	+3	+4
-19	+5	-3	-4
+19	-5	-3	+4
-19	-5	+3	-4

The divider is an iterative circuit that generates 1 quotient bit per cycle. It is implemented with a single 34-bit subtracter (really an adder with a conditional carry in) that adds or subtracts the divisor from the partial remainder every cycle using a non-restoring division algorithm for signed quantities. The 33rd bit distinguishes DIV from DIVU by either zero extending or sign extending the 32 bit operands. The 34th bit is needed to correctly determine the sign of the subtracter output.

Thus the pipeline flow of a division instruction and the most efficient subsequent read of the quotient (using MFLO) is as shown in the following diagram, assuming that all the intervening instructions complete in one cycle. If the MFLO is issued earlier it will stall until the divide completes. Less than 34 instructions may be issued if some of them take more than one cycle to complete (due to cache misses or data dependent stalls, for example).

```

DIV   I   S   E   D1 D2 D3 ... D33 D34
...
34 instructions
...
MFLO                                     I   S   E   M   W

```

Chapter

10

LX4280 Enhancements and Differences

10.1 Instruction Set

10.1.1 Conditional Move

The LX4280 introduces new instructions (MV<COND>) to support conditional operations. These instructions are described in this section.

Furthermore, the effect of any conditional execution can be “emulated” in the LX4280 with a sequence of two instructions by using the conditional move. For example:

Processor with conditional execution:

Inst 1: ALU operation sets condition flags
Inst 2: COND: ALU operation

LX4280:

Inst 1: ALU operation updates register rB (condition setting operation)
Inst 2: ALU operation with result directed to temp register rA
Inst 3: CMV<COND> rD, rA, rB

If rB satisfies the COND, rD is updated with rA; i.e. the 2nd ALU operation is executed to “completion”. Note that this sequence is interruptible. The if-then-else construct:

```

if (rB COND)
    rD = rA
else
    rD = rC

```

can be coded if the previous example is prefaced with:

```

MOVE rD, rC          // move rC to rD

```

One reason Lexra has provided conditional move is to facilitate initial porting of assembler code from processors with conditional execution to the LX4280.

Conditional Move Instructions

Instruction	Description
MOVZ rD, rS, rT	Conditional Move on Equal Zero If the general register rT is equal to 0, the general register rD is updated with rS; otherwise rD is unchanged.
MOVN rD, rS, rT	Conditional Move on Not Equal Zero If the general register rT is not equal to 0, the general register rD is updated with rS; otherwise rD is unchanged.

Nomenclature:

```

rD    =    r0 - r31
rS    =    r0 - r31
rT    =    r0 - r31

```

Usage Note: When combined with the SLT instruction, the conditional move instructions can be used to construct a complete set of conditional move macro-operations. For example:

```

if ( r3 < r4 )    r1 <-- r2

```

```

MVLT    r1, r2, r3, r4    ==>    SLT        AT, r3, r4
                                MOVN       r1, r2, AT

```

```

if ( r3 >= r4 )    r1 <-- r2

```

```

MVGE    r1, r2, r3, r4    ==>    SLT        AT, r3, r4
                                MOVZ       r1, r2, AT

```

MVLE	r1,r2,r3,r4	==>	SLT	AT,r4,r3
			MOVZ	r1,r2,AT

MVGT	r1,r2,r3,r4	==>	SLT	AT,r4,r3
			MOVN	r1,r2,AT

Mnemonic	31 - 26	25 - 21	20 - 16	15 - 11	10 - 6	5 - 0
MOVZ	000000	rS	rT	rD	00000	001010
MOVN	000000	rS	rT	rD	00000	001011

The LX4280 introduces eight (8) new low-overhead hardware interrupt signals. These signals are upward-compatible with the R3000 Exception Processing model and are useful for real-time applications.

Mnemonic	31 - 26	25 - 21	20 - 16	15 - 11	10 - 0
MFLXC0	010_000	00011	rT	rD	000 0000 0000
MTLXC0	010 000	00111	rT	rD	000 0000 0000

Selects general register r0 - r31.

rd

Selects Lexra Coprocessor0 register:

00000 ESTATUS
 00001 ECAUSE
 00010 INTVEC
 00011 reserved
 001xx reserved
 01xxx reserved
 1xxxx reserved

The new Lexra Coprocessor 0 registers are: ESTATUS (0), ECAUSE (1) and INTVEC (2):

ESTATUS (LX COP0 reg 0) Read/Write

31-24	23-16	15-0
0	IM[15-8]	0

ECAUSE (LX COP0 reg 1) Read-only

31-24	23-16	15-0
0	IP[15-8]	0

INTVEC (LX COP0 reg 2) Read/Write

31-6	5-0
BASE	0

ESTATUS contains the mask bits IM[15-8] for the new interrupt signals. IM[15-8] is reset to 0 so that, regardless of the global interrupt enable, IEc, none of the new interrupts will be activated. IP[15-8] for the new interrupt signals is located in ECAUSE and is Read-only. Each of these fields are similar to IM and IP defined in the R3000 Exception Processing model. The difference, however, is that the new interrupts are prioritized in hardware and each have a dedicated Exception Vector.

IP[15] has the highest interrupt priority, IP[14] next, etc. All new interrupts are higher priority than IP[0-7]. The BASE of the Exception vectors for the new interrupts is located in INTVEC and is program-defined. The particular vector for IP[15] is BASE

|| 111000, for IP[14] is BASE || 110000, etc. Two instructions can be executed at each vector; typically these will consist of a jump and its delay slot, with the target of the jump being either a shared interrupt handler or one that is unique to the particular interrupt.

Note that when a vectored interrupt causes an exception, all of the standard actions for an exception occur. These include the update of the EPC register and certain subfields of the standard STATUS and CAUSE registers. In particular, the Exception Code field of CAUSE indicates “Interrupt” and the “current” and “previous” Mode bits of the STATUS register are updated in the usual way.

1. if (IM[15-00] & IP[15-00]) $\neq 0^{16}$ and lec $\neq 0$

Interrupt request $\leftarrow 1$

else

Interrupt request $\leftarrow 0$.

2. if no higher priority Exception is active, an Int Exception is taken

ExcCode $\leftarrow 0$ (Int)

EPC $\leftarrow$ return PC

if IM[15] & IP[15] PC $\leftarrow$ BASE || 111000

else

if IM[14] & IP[14] PC $\leftarrow$ BASE || 110000

•

•

else

if IM[08] & IP[08] PC $\leftarrow$ BASE || 000000

else

PC $\leftarrow$ R3000 Common Exception Vector

10.1.3 32x32 MADD(U), MSUB(U)

32-bit x 32-bit Multiply-Accumulate instructions MAD(U) and MSUB(U) are implemented using the 16-bit Multiply-Accumulate datapath and a special adder. It provides a 64-bit result. A Multiply-Accumulate operation can be issued every four

cycles, while the result is available in six cycles. The least significant 32 bits of the sum are available one cycle earlier than the most significant 32 bits.

32-bit x 32-bit Multiply-Accumulate instructions Instruction

Instruction	Description
MAD rS, rT	The contents of rS are multiplied by rT, treating the operands as signed 2's complement values. The 64-bit product is added to the concatenation HI LO to form a 64-bit result ignoring any overflow. The upper 32-bits of the 64-bit result are stored in the HI register. The lower 32-bits are stored in the LO register. $t \leftarrow (HI \parallel LO) + Rs * Rt$ $LO \leftarrow t[31:0]$ $HI \leftarrow t[63:32]$
MADU rS, rT	The contents of rS are multiplied by rT, treating the operands as unsigned values. The 64-bit product is added to the concatenation HI LO to form a 64-bit result ignoring any overflow. The upper 32-bits of the 64-bit result are stored in the HI register. The lower 32-bits are stored in the LO register. $t \leftarrow (HI \parallel LO) + (0 \parallel Rs) * (0 \parallel Rt)$ $LO \leftarrow t[31:0]$ $HI \leftarrow t[63:32]$
MSUB rS, rT	The contents of rS are multiplied by rT, treating the operands as signed 2's complement values. The 64-bit product is subtracted from the concatenation HI LO to form a 64-bit result ignoring any overflow. The upper 32-bits of the 64-bit result are stored in the HI register. The lower 32-bits are stored in the LO register. $t \leftarrow (HI \parallel LO) - Rs * Rt$ $LO \leftarrow t[31:0]$ $HI \leftarrow t[63:32]$
MSUBU rS, rT	The contents of rS are multiplied by rT, treating the operands as unsigned values. The 64-bit product is subtracted from the concatenation HI LO to form a 64-bit result ignoring any overflow. The upper 32-bits of the 64-bit result are stored in the HI register. The lower 32-bits are stored in the LO register. $t \leftarrow (HI \parallel LO) - (0 \parallel Rs) * (0 \parallel Rt)$ $LO \leftarrow t[31:0]$ $HI \leftarrow t[63:32]$

32-bit x 32-bit Multiply-Accumulate instructions Instruction Encodings

Mnemonic	31 - 26	25 - 21	20 - 16	15 - 6	5 - 0
MAD rS, rT	011100	rS	rT	0000000000	00000
MADU rS, rT	011100	rS	rT	0000000000	00001
MSUB rS, rT	011100	rS	rT	0000000000	00100
MSUBU rS, rT	011100	rS	rT	0000000000	00101

10.2 Pipeline

10.2.1 Six Stage Pipeline

The LX4280 issues dual 32-bit instructions to two distinct 6-stage execution pipelines. *Superscalar* architectures have been widely deployed in RISC CPUs where increased performance is obtained at the cost of increased area.

Compared to the conventional five stage pipeline employed by the LX4180, the LX4280 adds one additional stage, the D (Decode) stage.

LX4280 Pipeline:

Stage 1 I	Instruction fetch
Stage 2 D	Decode
Stage 3 S	Source fetch
Stage 4 E	Execution
Stage 5 M	Memory data select
Stage 6 W	Write back to register file

The LX4280 I-Cache and IRAM can fetch two 32-bit instructions simultaneously. Each instruction can issue to either Pipe A or Pipe B, depending on which Pipe supports the required resources for execution. For example, load/store instructions are only supported in Pipe A, MAC operations only in Pipe B. All ALU instructions are supported in both Pipe A and Pipe B. Data dependencies between the two instructions will prevent dual issue. Customers who wish to optimize LX4280 assembly programming will need to be familiar with the dual issue rules described in section 10.2.3. For most customers, the C compiler will be sufficient. Correct operation, whether in C programming or assembly programming, is always assured through built-in hardware interlocks.

To avoid degrading performance, the superscalar issue logic operates during the Decode-Stage of the pipeline which occurs after I (Instruction Fetch) and before S (Source Fetch). The D stage allows the system clock cycle time to be not only

maintained, but improved, compared to the five stage pipeline. Support for fully synchronous instruction memories in the LX4280 has the added benefit of isolating the processor logic from the customer-supplied memories, thus facilitating integration of the LX4280 into SoC designs.

As a result of the D-Stage, a two cycle penalty is incurred on branch prediction failure vs. the one-cycle penalty in the LX4180 five stage pipeline.

10.2.2 Superscalar RALU

Operations are divided between Pipe A and Pipe B in such a way that the RALU is the only major section of the processor which requires both Pipe A and B instructions. Coprocessor 0, as well as the customer-defined Coprocessors 1-3, only require the Pipe A instruction.

To “first approximation” the superscalar RALU is a “doubling” of the LX4180 RALU: it includes an 8-port (4r/4w) general register file with 4-ports (2r/2w) assigned to Pipe A, and 4-ports (2r/2w) assigned to Pipe B. In each Pipe, one write port is dedicated to register file updates from the Data Bus (Loads, MFCz, CFCz - moves from Coprocessor). The remaining three ports (2r/1w) are available for the other operations assigned to that Pipe. As a result, Loads can dual-issue with any MAC or ALU instruction without register port access restriction.

Each Pipe has an ALU and a nearly-independent control section. Differences occur in the assignment of operations to Pipe A and Pipe B, and in the pipeline features to support superscalar. The pipeline differences in the RALU to support superscalar issue are:

- Data must be forwarded from Pipe A (Pipe B) to Pipe B (Pipe A) when the input to a Pipe B (Pipe A) execution unit requires a result computed earlier in Pipe A (Pipe B).
- If both Pipe A and Pipe B operations write the same register, the RALU suppresses the write for the earlier instruction based on program order.

10.2.3 Assignment of Instructions to Pipe A, Pipe B.

The Table lists the detailed assignment of instructions to Pipe A and Pipe B. Pipe B is called the “MAC Pipe” because it uniquely supports multiply-accumulate, as well as multiply and divide operations. The MAC unit, which is attached to Pipe B as Custom Engine 0 (CE0), includes the accumulator registers (including HI and LO) and therefore also supports the *move to* and *move from* operations which transfer data between these registers and the general register file.

Pipe A is called the “Load/Store Pipe” because it uniquely supports the Load and Store operations.

The Coprocessor operations, and all “sequencing control instructions” (branches, jumps) are unique to Pipe A. As a result, Pipe B instructions are not routed to Coprocessors.

The opcodes reserved for a customer defined Custom Engine 1 (CE1) are routed to Pipe B, since CE1 is attached to Pipe B.

Assignment of Instructions of Pipe A, Pipe B

	Pipe A	Pipe B
	The Load / Store Pipe	The MAC Pipe
MIPS 32-bit General Instructions	MIPS 32-bit General Instructions except: CE1 Custom Engine Opcodes, MULT(U), DIV(U), MFHI, MFLO, MTHI, MTLO,MAD(U),MSUB(U)	CE1 Custom Engine Opcodes, MIPS 32-bit ALU Instructions Note: No Load or Store Instructions [MULT(U), DIV(U), MFHI, MFLO, MTHI, MTLO,MAD(U),MSUB(U) – if optional MAC is installed]
MIPS 32-bit Control Instructions	J, JAL, JR, JALR, JALX SYSCALL, BREAK, All Branch Instructions, All COPz, SWCz, LWCz	
MIPS16 Instructions (No Doubleword Instructions)	All MIPS16 Instructions except: MULT(U), DIV(U), MFHI, MFLO	MULT(U), DIV(U), MFHI, MFLO
EJTAG Instructions	DERET, SDBBP (including MIPS16 SDBBP)	
Lexra Control Instructions	MTRU, MFRU, MTRK, MFRK, MTLXC0,MFLXC0	

10.2.4 MIPS16

If MIPS16 code compression has been enabled, each 32-bit fetch is interpreted as a pair of 16-bit instructions encoded according to the MIPS16 Specification. MIPS16 instructions are not dual-issued, but always issued to Pipe A. It is expected that MIPS16 code compression is enabled for “outer loop” code where code density is more important than performance. The critical Register File read addresses for MIPS16 are resolved during the D-stage so that register file access for MIPS16 instructions, as for 32-bit MIPS instructions, can begin on the rising edge of the S-Stage clock.

10.3 EJTAG

10.3.1 Dual Pipe PC Trace

The EJTAG PC Trace facility specifies that a PCST code is issued every cycle to indicate if the instruction pipeline has stalled, sequentially completed an instruction, or taken a branch or jump. In order to accommodate the two pipelines in the LX4280, the capability of emitting more than one PCST code per cycle is employed. Specifically, to the external EJTAG logic, the LX4280 appears to be a single pipe machine running at twice the speed that it actually does.

Since there must be an even number of PCST codes made available at every DCLK (in the EJTAG nomenclature), the DCLK parameter "N" must be set to 2 or 4 according to the ratio of DCLK period to one-half the actual SYSCLK period of the pipeline clock.

Furthermore, this implies that until the EJTAG specification is extended beyond N=4 or DCLK maximum frequency of 100MHz, the PCST feature can be used with a maximum SYSCLK frequency of 200MHz.

10.3.2 Vectored Interrupts and PC Trace

The EJTAG PC trace facility specifies a 3-bit code to be output serially on the TPC line(s) when an exception occurs. In order to distinguish the eight vectored interrupts in the LX4280 from all other exceptions, a 4-bit code is used instead.

For all exceptions *other* than vectored interrupts, the most significant bit of the 4-bit code is zero and the remaining 3-bits are the standard 3-bit code. Note that this includes the standard software and hardware interrupts numbered 0 through 7.

For vectored interrupts, the most significant bit is always 1. The 4-bit code is simply the number of the vectored interrupt (from 8 to 15) being taken.

Since the target of the vectored interrupt is determined by the contents of the INTVEC register, the debug software which monitors the EJTAG PC Trace codes must be aware of the contents of this register in order to trace the code after the vectored interrupt is taken.

For backward compatibility, the LX4280 can be configured to use only the 3-bit standard codes. In that case, if a vectored interrupt is taken, the RESET 3-bit code will be presented.

10.4 Cache

This section is intended to provide a list of Icache and Dcache features and configurations available in the LX4280, especially where those features or configurations differ from the LX4180.

- **Line Size** — The RTL product has a configurable line size that may be 4, 8, 16, or 32 words (the chosen size applies to both Icache and Dcache). The Smoothcore product has a fixed line size of 4 words.
- **Cache Size** — The Icache and Dcache are each independently configurable to the following sizes: 0K, 1K, 2K, 4K, 8K, 16K, 32K or 64K bytes. Large cache sizes are more likely to impact the cycle time, especially for the Dcache.
- **Dcache Associativity** — The Dcache is always direct mapped.
- **Icache Associativity** — The Icache is configurable as either direct mapped or two-way set associative.
- **Program Controlled Cache Invalidate** — this feature is the same as in LX4180.
- **Program Controlled Icache Locking** — this feature is the same as in LX4180, requiring a 2-way set associative Icache.
- **Scratchpad Local Memory Size** — The so-called IRAM and DRAM are supported in sizes up to 512K bytes each. Large sizes will impact the cycle time.
- **DMA Access to caches and local memory** — Supported as in LX4180.
- **System Bus** — the caches and local memories have 64-bit data paths, while the System Bus currently remains at 32-bits width, with the same CBUS/LBUS options as in LX4180.

10.5 Improved DIV(U) Performance

The divide operation in the LX420 completes in 19 cycles instead of 34 for the LX4180.

Thus the pipeline flow of an LX4280 division instruction and the most efficient subsequent read of the quotient (using MFLO) is as shown in the following diagram, assuming that all the intervening instructions complete in one cycle. If the MFLO is issued earlier it will stall until the divide completes. Less than 19 instructions may be issued if some of them take more than one cycle to complete (due to cache misses or data dependent stalls, for example).

DIV	I	S	E	D1	D2	D3	...	D18	D19	
...										
19 instructions										
...										
MFLO									I	S E M W